

CSES PROBLEM SET

Sorting & Searching

All 35 problems explained intuition-first: every idea is built up step by step before any code appears, and each problem comes with more than one way to solve it, in C++.

35

problems

104

tested programs

1

template

Contents

1. Distinct Numbers
2. Apartments
3. Ferris Wheel
4. Concert Tickets
5. Restaurant Customers
6. Movie Festival
7. Sum of Two Values
8. Maximum Subarray Sum
9. Stick Lengths
10. Missing Coin Sum
11. Collecting Numbers
12. Collecting Numbers II
13. Playlist
14. Towers
15. Traffic Lights
16. Distinct Values Subarrays
17. Distinct Values Subsequences
18. Josephus Problem I
19. Josephus Problem II
20. Nested Ranges Check
21. Nested Ranges Count
22. Room Allocation
23. Factory Machines
24. Tasks and Deadlines
25. Reading Books
26. Sum of Three Values
27. Sum of Four Values
28. Nearest Smaller Values
29. Subarray Sums I
30. Subarray Sums II
31. Subarray Divisibility
32. Distinct Values Subarrays II
33. Array Division
34. Movie Festival II
35. Maximum Subarray Sum II

How to use this book

Every chapter follows the same path, the way you would think in a contest:

1. **The task.** The problem statement cut down to its core, plus the constraints that decide which complexity is fast enough.
2. **Building the intuition.** A chain of small observations, each following from the one before, until the solution feels obvious. Try to stop after each step and guess the next one yourself.
3. **Approaches.** We start from the brute force (so you see exactly what is slow) and move to the optimal solution. Where a genuinely different technique also works, it follows as an alternative.
4. **Complexity & pitfalls.** The time and memory of each approach, and the traps that cost people accepted verdicts.

NOTE

Every program in this book was compiled with `g++ -O2 -std=c++17` inside your template and **stress-tested**: 300 random small tests compared against a brute-force solution, plus maximum-size inputs to check the running time. What you see is exactly the code that passed.

Your template

All solutions are written for this template. To keep the pages readable, **each code block shows only what goes in place of `solve()`** (plus any helper functions or structs, which go just above `solve()`).

Everything else stays exactly as below.

```
#include <bits/stdc++.h>
using namespace std;

#define int long long int
#define Start() ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0); clock_t z = clock();
#define bug(...) __f (#__VA_ARGS__, __VA_ARGS__)
#define print(a) for(auto x : a) cout << x << " "; cout << '\n'
#define print1(a) for(auto x : a) cout << x.first << " " << x.second << '\n'
#define print2(a) for (int i = 0; i < (int)a.size(); i++){print(a[i]);}
#define End() cerr << fixed << "Run Time : " << ((double)(clock() - z) / CLOCKS_PER_SEC);

template <typename Arg1>
void __f (const char* name, Arg1&& arg1) { cout << name << " : " << arg1 << endl; }
template <typename Arg1, typename... Args>
void __f (const char* names, Arg1&& arg1, Args&&... args){
    const char* comma = strchr (names + 1, ',');
    cout.write (names, comma - names) << " : " << arg1 << " | "; __f (comma + 1, args...);
}

const int mod = 1e9+7;
const int N = 100005;

void solve() {

}

int32_t main(){
    Start();
    int t = 1;
    //cin >> t;
```

```

while (t--) solve();
End();
return 0;
}

```

A few consequences of the template that the solutions rely on:

- `#define int long long` makes every `int` a 64-bit integer. Sums like $2 \cdot 10^5 \times 10^9 = 2 \cdot 10^{14}$ fit without any extra care. Where you see `LLONG_MAX` / `LLONG_MIN`, that is why.
- `main` is declared as `int32_t main()` because `int` itself has been redefined.
- `print(a)` prints a container on one line; several solutions use it for the output.
- `End()` writes the run time to `stderr`. The judge ignores `stderr`, so it is safe to submit.
- `mod = 1e9+7` is used directly by *Distinct Values Subsequences*.

WATCH OUT

Policy-based data structures (`ordered_set`). Two solutions use GNU's `tree<>`. Its headers must be included **above** `#define int long long int`; otherwise the macro rewrites `int` inside the library headers and compilation fails. Those code blocks are marked with (1) *put above the define* and (2) *replaces solve()*.

The techniques in this section

Technique	Problems
Sort, then a greedy decision	Apartments, Ferris Wheel, Movie Festival, Stick Lengths, Missing Coin Sum, Tasks and Deadlines, Reading Books
Two pointers / sliding window	Apartments, Ferris Wheel, Sum of Two/Three/Four Values, Playlist, Subarray Sums I, Distinct Values Subarrays I & II
<code>set</code> / <code>multiset</code> as a sorted, changing collection	Concert Tickets, Towers, Traffic Lights, Room Allocation, Movie Festival II, Maximum Subarray Sum II
Sweep line over events	Restaurant Customers, Room Allocation, Nested Ranges Check/Count
Prefix sums + counting	Maximum Subarray Sum, Subarray Sums II, Subarray Divisibility, Maximum Subarray Sum II
Binary search on the answer	Factory Machines, Array Division
Monotonic stack / deque	Nearest Smaller Values, Maximum Subarray Sum II
Order statistics (Fenwick tree, <code>ordered_set</code>)	Josephus Problem II, Nested Ranges Count
Positions of values instead of values at positions	Collecting Numbers I & II
Counting / combinatorics	Distinct Values Subsequences, Subarray Divisibility

NOTE

A rule of thumb used throughout. A judge runs roughly 10^8 simple operations per second. With n up to $2 \cdot 10^5$, $O(n^2)$ is about $4 \cdot 10^{10}$ operations, which is far too slow, while $O(n \log n)$ is about $3.5 \cdot 10^6$, which is instant. So almost every problem here asks the same question: *how do I avoid looking at all pairs?*

1 • Distinct Numbers

cses.fi/problemset/task/1621 · Pattern: **sorting, sets, hashing**

The task

You get n integers. Count how many **different** values appear among them.

- $n \leq 2 \cdot 10^5$, values up to 10^9 , so an $O(n \log n)$ solution is needed.
- Values are too large to use directly as array indices (no `bool seen[1e9]`).

Building the intuition

Step 1: say what we count, precisely. A value contributes 1 to the answer the *first time* it appears and nothing after that. So the question for every element is simply: *have I seen this value before?*

Step 2: the obvious way is too slow. Answering “seen before?” by scanning all earlier elements costs $O(n)$ per element, $O(n^2)$ in total. With $n = 2 \cdot 10^5$ that is about $2 \cdot 10^{10}$ comparisons, far too many.

Step 3: make “seen before?” cheap. There are two classic ways:

- **Put equal values next to each other.** After sorting, all copies of a value form one block, so “seen before” just means “equal to my left neighbour”. We count the places where a block *starts*.
- **Remember what we’ve seen** in a structure with fast lookups: a `set` ($O(\log n)$) or a hash set ($O(1)$ on average).

WORKED EXAMPLE

```
input:  2 3 2 2 3
sorted: 2 2 2 3 3
        ^   ^
        a new block starts at index 0 and at index 3 -> answer 2
```

KEY IDEA

Sorting turns a global question (“does this value appear anywhere else?”) into a **local** one (“is it equal to its neighbour?”). You will see this move again and again in this section.

Approach 1 • Brute force, $O(n^2)$

For each element, look back at everything before it.

```
void solve() {
    int n; cin >> n;
    vector<int> a(n);
    for (auto &x : a) cin >> x;
    int distinct = 0;
    for (int i = 0; i < n; i++) {
        bool seenBefore = false;
        for (int j = 0; j < i; j++)
            if (a[j] == a[i]) seenBefore = true;
    }
}
```

```

        if (!seenBefore) distinct++;
    }
    cout << distinct << '\n';
}

```

Approach 2 • Sort and count block starts, $O(n \log n)$

```

void solve() {
    int n; cin >> n;
    vector<int> a(n);
    for (auto &x : a) cin >> x;

    sort(a.begin(), a.end());           // equal values become neighbours

    int distinct = 1;                    // a[0] always starts a new group
    for (int i = 1; i < n; i++)
        if (a[i] != a[i - 1]) distinct++; // a new value starts here

    cout << distinct << '\n';
}

```

A popular one-liner does the same thing: `sort(a.begin(), a.end()); cout << unique(a.begin(), a.end()) - a.begin();`. The `unique` function moves one copy of each block to the front and returns the end of that part.

Approach 3 • `std::set`, $O(n \log n)$

A `set` keeps each value only once, so its final size is the answer. This needs less thought but is about 3× slower than sorting a vector, because every insert allocates a tree node.

```

void solve() {
    int n; cin >> n;
    set<int> st;                               // a set silently ignores duplicates
    for (int i = 0; i < n; i++) {
        int x; cin >> x;
        st.insert(x);
    }
    cout << st.size() << '\n';
}

```

Approach 4 • Hash set with a safe hash, $O(n)$ expected

`unordered_set` answers “seen?” in $O(1)$ on average. The default hash for integers is the identity, however, and tests can be built so that every value lands in the same bucket, which makes each operation $O(n)$. A randomized hash (splitmix64 with a time-based seed) removes that risk. This `custom_hash` is worth keeping in your library, since it is reused later in this book.

```

// splitmix64: a random-seeded hash so anti-hash tests can't slow us down
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {

```

```

        static const uint64_t SEED = chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + SEED);
    }
};

void solve() {
    int n; cin >> n;
    unordered_set<int, custom_hash> st;
    st.reserve(2 * n);
    for (int i = 0; i < n; i++) {
        int x; cin >> x;
        st.insert(x);
    }
    cout << st.size() << '\n';
}

```

Complexity & pitfalls

Approach	Time	Memory
Brute force	$O(n^2)$	$O(n)$
Sort + count	$O(n \log n)$	$O(n)$
set	$O(n \log n)$	$O(n)$
Hash set (custom hash)	$O(n)$ expected	$O(n)$

WATCH OUT

- `unordered_set<int>` with the **default** hash can be forced into its worst case. Use the sort solution or a custom hash.
- Don't try `bool seen[1000000001]` : that is about 1 GB of memory.

2 • Apartments

cses.fi/problemset/task/1084 · Pattern: **sort both sides + two pointers (greedy matching)**

The task

There are n applicants and m apartments. Applicant i wants size a_i but accepts any apartment whose size is in $[a_i - k, a_i + k]$. Each applicant gets at most one apartment and each apartment goes to at most one applicant. Maximize the number of happy applicants.

- $n, m \leq 2 \cdot 10^5$, sizes and k up to 10^9 .

Building the intuition

Step 1: redraw the problem. Every applicant is an **interval** $[a - k, a + k]$ on the number line and every apartment is a **point**. We want to match as many points to intervals as possible, where a point can only go to an interval that contains it.

Step 2: all intervals have the same length $2k$. So if we sort applicants by a , their intervals are sorted by left end *and* by right end at the same time. That is what makes a simple greedy work.

Step 3: look at the smallest things first. Take the smallest remaining applicant (interval $[a - k, a + k]$) and the smallest remaining apartment b . Three things can happen:

Situation	Meaning	What to do
$b < a - k$	b is too small for this applicant, and every other applicant wants even more	throw the apartment away
$b > a + k$	every remaining apartment is $\geq b$, so all are too big for this applicant	this applicant can never be served, skip him
otherwise	b fits	match them

Step 4: why is matching them safe? This applicant's interval ends earliest of all remaining intervals, so he is the most "urgent". Giving him the *smallest* usable apartment keeps every larger apartment available for applicants who may need it. If an optimal answer did it differently, we could swap partners without losing a match (the exchange argument).

WORKED EXAMPLE

$k = 5$, desired sizes 60 45 80 60, apartments 30 60 75.

```
sorted a: 45 60 60 80      sorted b: 30 60 75
a=45 [40,50], b=30 -> 30 < 40: apartment useless      -> next apartment
a=45 [40,50], b=60 -> 60 > 50: applicant 45 hopeless    -> next applicant
a=60 [55,65], b=60 -> fits! matched = 1
a=60 [55,65], b=75 -> 75 > 65: applicant hopeless      -> next applicant
a=80 [75,85], b=75 -> fits! matched = 2                answer: 2
```

Approach 1 · Brute force (exponential)

Trying every assignment is only possible for tiny inputs (a bitmask DP over apartments, $O(n \cdot m \cdot 2^m)$). Its real use is as a checker: it is how the greedy below was validated on thousands of random tests.

Approach 2 · Sort both + two pointers, $O(n \log n + m \log m)$

```
void solve() {
    int n, m, k; cin >> n >> m >> k;
    vector<int> a(n), b(m);
    for (auto &x : a) cin >> x; // desired sizes
    for (auto &x : b) cin >> x; // apartment sizes
    sort(a.begin(), a.end());
    sort(b.begin(), b.end());

    int i = 0, j = 0, matched = 0;
    while (i < n && j < m) {
        if (b[j] < a[i] - k) j++; // apartment too small for everyone left -> discard it
        else if (b[j] > a[i] + k) i++; // applicant i can never be satisfied -> skip him
        else { matched++; i++; j++; } // fits: match them greedily
    }
    cout << matched << '\n';
}
```

Approach 3 · Multiset lookup, $O((n + m) \log m)$

Another way to phrase the same greedy: process applicants from small to large and give each one the **smallest free apartment that is $\geq a - k$** , if that apartment is also $\leq a + k$. A `multiset` finds it with `lower_bound`.

```
void solve() {
    int n, m, k; cin >> n >> m >> k;
    vector<int> a(n);
    multiset<int> free_apts;
    for (auto &x : a) cin >> x;
    for (int j = 0; j < m; j++) { int x; cin >> x; free_apts.insert(x); }
    sort(a.begin(), a.end());

    int matched = 0;
    for (int want : a) {
        // smallest free apartment that is >= want - k
        auto it = free_apts.lower_bound(want - k);
        if (it != free_apts.end() && *it <= want + k) {
            matched++;
            free_apts.erase(it); // erase ONE copy (by iterator), not all equal values
        }
    }
    cout << matched << '\n';
}
```

KEY IDEA

The two-pointer version and the multiset version make exactly the same decisions. The two-pointer version is simply faster because it never deletes anything; it only moves forward.

Complexity & pitfalls

Approach	Time	Memory
Two pointers	$O(n \log n + m \log m)$	$O(n + m)$
Multiset	$O((n + m) \log m)$	$O(m)$

WATCH OUT

- `multiset::erase(value)` removes **all** copies of that value. Always erase through an iterator: `ms.erase(it)`.
- Comparing `b < a - k` is safe here, because `int` is `long long` in the template.

3 • Ferris Wheel

cses.fi/problemset/task/1090 · Pattern: **sort + two pointers (greedy pairing)**

The task

n children with weights p_i . A gondola holds **one or two** children and their total weight must be $\leq x$. What is the minimum number of gondolas? (Every $p_i \leq x$, so each child fits alone.)

- $n \leq 2 \cdot 10^5$.

Building the intuition

Step 1: each gondola holds at most 2. So the answer is $n - (\text{number of pairs we form})$, and our goal is to form as many valid pairs as possible.

Step 2: focus on the hardest child, the heaviest one. He needs a gondola either way. Who should share it with him?

Step 3: the best partner for the heaviest child is the lightest child.

- If even the lightest child can't ride with him, **nobody** can, so the heaviest child rides alone.
- If the lightest one fits, pair them. Could that be a mistake? Suppose an optimal answer pairs the heaviest H with some Y , and the lightest L with some Z . Rearrange into (H, L) and (Y, Z) . The pair (H, L) fits, as we just checked. The pair (Y, Z) also fits, because $Z \leq L$ gives $Y + Z \leq Y + L \leq x$. The number of gondolas stays the same. (If H or L rode alone in that answer, the rearrangement only gets better.) So pairing H with L is always safe.

Step 4: repeat on what's left. After sorting, the lightest child is at pointer l and the heaviest at pointer r . Every step uses up r , and sometimes also l .

WORKED EXAMPLE

$x = 10$, weights 7 2 3 9, sorted 2 3 7 9.

$l=2, r=9 : 2+9 = 11 > 10 \rightarrow 9$ rides alone	$\text{gondolas} = 1$
$l=2, r=7 : 2+7 = 9 \leq 10 \rightarrow 2$ and 7 together	$\text{gondolas} = 2$
$l=3, r=3 : \text{same child} \rightarrow 3$ rides alone	$\text{gondolas} = 3$

Approach 1 • Brute force

A bitmask DP over the set of children still waiting tries every possible pairing. It is exponential and only useful for checking.

Approach 2 • Sort + two pointers, $O(n \log n)$

```
void solve() {
    int n, x; cin >> n >> x;
    vector<int> p(n);
    for (auto &v : p) cin >> v;
    sort(p.begin(), p.end());
```

```

int l = 0, r = n - 1, gondolas = 0;
while (l <= r) {
    // the heaviest child p[r] always takes a gondola now;
    // the lightest child joins him if they fit together
    if (l < r && p[l] + p[r] <= x) l++;
    r--;
    gondolas++;
}
cout << gondolas << '\n';
}

```

Approach 3 · Lightest child + best-fitting partner (multiset), $O(n \log n)$

The mirror-image greedy also works: take the **lightest** child and pair them with the **heaviest child that still fits**, which is `upper_bound(x - light)` minus one in a multiset. Intuitively, the lightest child is the most “flexible”, so we use that flexibility on the heaviest partner it can carry, the one who is hardest to place elsewhere.

```

void solve() {
    int n, x; cin >> n >> x;
    multiset<int> ms;
    for (int i = 0; i < n; i++) { int v; cin >> v; ms.insert(v); }

    int gondolas = 0;
    while (!ms.empty()) {
        // take the lightest remaining child...
        int light = *ms.begin();
        ms.erase(ms.begin());
        // ...and pair him with the HEAVIEST child that still fits
        auto it = ms.upper_bound(x - light);
        if (it != ms.begin()) ms.erase(prev(it));
        gondolas++;
    }
    cout << gondolas << '\n';
}

```

Complexity & pitfalls

Approach	Time	Memory
Two pointers	$O(n \log n)$	$O(n)$
Multiset	$O(n \log n)$	$O(n)$

WATCH OUT

- Use `l <= r` in the loop, not `l < r`, or the last child in the middle gets no gondola.
- Don't pair a child with themselves: the condition `l < r` inside the loop guards this.

4 • Concert Tickets

cses.fi/problemset/task/1091 · Pattern: **multiset**: “largest $\leq t$ ” with deletion

The task

n tickets with prices h_i . Customers arrive one by one, and customer j will pay at most t_j . Each customer gets the **most expensive ticket whose price is $\leq t_j$** , and that ticket is then sold. Print the price paid, or -1 if no ticket is cheap enough.

- $n, m \leq 2 \cdot 10^5$.

Building the intuition

Step 1: find the two operations we need. For every customer we must:

1. find the **largest price $\leq t$** among the tickets still available, and
2. **remove** that ticket.

Step 2: what’s slow about the obvious tools?

Structure	Find largest $\leq t$	Remove
unsorted array	$O(n)$	$O(1)$
sorted vector	$O(\log n)$ (binary search)	$O(n)$ (shifting)
balanced BST (multiset)	$O(\log n)$	$O(\log n)$

A **multiset** is a sorted collection that stays sorted while we delete from it, so it is the natural tool. It must be a *multiset* because prices repeat.

Step 3: how to get “largest $\leq t$ ” out of a multiset. C++ gives us `upper_bound(t)`, the first element that is $> t$. The element just before it is the largest one that is $\leq t$. If `upper_bound` returns `begin()`, there is nothing $\leq t$ at all.

```
tickets: 3 5 5 7 8      t = 6
          ^ upper_bound(6) points at 7
          ^ one step back: 5    <- the answer
```

WORKED EXAMPLE

Prices 5 3 7 8 5, customers 4 8 3.

```
t=4 : largest <= 4 is 3  -> print 3, remaining {5,5,7,8}
t=8 : largest <= 8 is 8  -> print 8, remaining {5,5,7}
t=3 : nothing <= 3      -> print -1
```

Approach 1 • Brute force, $O(n \cdot m)$

For each customer, scan all tickets for the best unsold one. That is up to $4 \cdot 10^{10}$ steps.

Approach 2 · Multiset, $O((n + m) \log n)$

```
void solve() {
    int n, m; cin >> n >> m;
    multiset<int> tickets;
    for (int i = 0; i < n; i++) { int h; cin >> h; tickets.insert(h); }

    for (int i = 0; i < m; i++) {
        int t; cin >> t;
        auto it = tickets.upper_bound(t);    // first ticket with price > t
        if (it == tickets.begin()) {        // nothing is <= t
            cout << -1 << '\n';
        } else {
            --it;                            // largest price <= t
            cout << *it << '\n';
            tickets.erase(it);               // erase exactly this one ticket
        }
    }
}
```

Approach 3 · Sorted array + “next free to the left” (DSU), almost $O((n + m) \log n)$

Here is a neat trick worth knowing. Sort the prices once. For a customer, binary search gives the last index `idx` with price $\leq t$. If that ticket is already sold, we want the nearest *unsold* ticket to its left.

Keep `par[i]`, which means “the nearest index $\leq i$ that might still be unsold”. When ticket `j` is sold, set `par[j] = j - 1`. Following `par` pointers finds the next free ticket, and **path compression** (as in a disjoint-set union) makes long chains of sold tickets collapse, so the total work stays almost linear. Index 0 is a sentinel meaning “none left”.

```
// Sorted prices + DSU "jump left": par[i] = nearest index <= i that is still unsold
// index 0 is a sentinel meaning "no ticket available"
vector<int> par;
int find(int x) { return par[x] == x ? x : par[x] = find(par[x]); }

void solve() {
    int n, m; cin >> n >> m;
    vector<int> h(n + 1);
    for (int i = 1; i <= n; i++) cin >> h[i];
    sort(h.begin() + 1, h.end());
    par.resize(n + 1);
    iota(par.begin(), par.end(), 0);

    for (int i = 0; i < m; i++) {
        int t; cin >> t;
        // idx = number of prices <= t -> the last such index in 1-based h
        int idx = upper_bound(h.begin() + 1, h.end(), t) - h.begin() - 1;
        int j = find(idx);                // nearest unsold at or to the left
        if (j == 0) cout << -1 << '\n';
        else {
            cout << h[j] << '\n';
            par[j] = j - 1;                // sold: future searches jump past it
        }
    }
}
```

KEY IDEA

“Delete an element and then find the nearest surviving element to the left or right” is a general pattern. The DSU-with-path-compression trick solves it on any sorted array, and it comes back in harder problems (painting segments, scheduling with deadlines).

Complexity & pitfalls

Approach	Time	Memory
Brute force	$O(n \cdot m)$	$O(n)$
Multiset	$O((n + m) \log n)$	$O(n)$
Sorted + DSU	$O(n \log n + m \log n)$	$O(n)$

WATCH OUT

- `tickets.erase(*it)` or `tickets.erase(value)` would remove **every** ticket with that price. Use `tickets.erase(it)`.
- Never decrement `begin()`: check `it == tickets.begin()` first.

5 • Restaurant Customers

cses.fi/problemset/task/1619 · Pattern: **sweep line over events**

The task

n customers, each with an arrival time a and a leaving time b . What is the **maximum number of customers in the restaurant at the same time**? All $2n$ times are distinct.

- $n \leq 2 \cdot 10^5$, times up to 10^9 .

Building the intuition

Step 1: when can the count change? Only at the moment somebody arrives (+1) or leaves (−1). Between two consecutive events, nothing happens.

Step 2: so only $2n$ moments matter. Times go up to 10^9 , but we don't care about the timeline itself, only the **order** of the events.

Step 3: sweep. Turn each customer into two events, $(a, +1)$ and $(b, -1)$. Sort all events by time and walk through them with a running counter. The largest value the counter reaches is the answer. This is the **sweep line** technique: imagine a vertical line moving left to right through time and updating a counter at each event.

WORKED EXAMPLE

Customers (5,8), (2,4), (3,9).

time :	2	3	4	5	8	9	
event:	+1	+1	-1	+1	-1	-1	
count:	1	2	1	2	1	0	maximum = 2

NOTE

All times are distinct, so we don't need to decide whether a leave and an arrival at the *same* moment overlap. In problems where they can coincide (e.g. Room Allocation), the tie-break between the two events matters.

Approach 1 • Brute force, $O(n^2)$

The maximum is always reached right at some arrival. For each arrival moment, count the customers present at that moment.

```
void solve() {
    int n; cin >> n;
    vector<int> a(n), b(n);
    for (int i = 0; i < n; i++) cin >> a[i] >> b[i];
    int best = 0;
    // the maximum is always reached right at some arrival moment
    for (int i = 0; i < n; i++) {
        int cnt = 0;
        for (int j = 0; j < n; j++) if (a[j] <= a[i] && a[i] < b[j]) cnt++;
    }
}
```

```

        best = max(best, cnt);
    }
    cout << best << '\n';
}

```

Approach 2 · Sorted events, $O(n \log n)$

```

void solve() {
    int n; cin >> n;
    vector<pair<int,int>> events;           // (time, +1 arrive / -1 leave)
    for (int i = 0; i < n; i++) {
        int a, b; cin >> a >> b;
        events.push_back({a, +1});
        events.push_back({b, -1});
    }
    sort(events.begin(), events.end());    // times are distinct, so no tie issues

    int cur = 0, best = 0;
    for (auto [time, delta] : events) {
        cur += delta;
        best = max(best, cur);
    }
    cout << best << '\n';
}

```

Approach 3 · Two sorted lists, merged, $O(n \log n)$

We don't even need to know which arrival belongs to which departure. Sort arrivals and departures **separately**, then merge them like in merge sort: whichever of the next arrival and the next departure comes first is the next event.

```

void solve() {
    int n; cin >> n;
    vector<int> in(n), out(n);
    for (int i = 0; i < n; i++) cin >> in[i] >> out[i];
    sort(in.begin(), in.end());
    sort(out.begin(), out.end());

    // merge two sorted lists: whichever event comes first happens first
    int i = 0, j = 0, cur = 0, best = 0;
    while (i < n) {
        if (in[i] < out[j]) { cur++; i++; best = max(best, cur); }
        else { cur--; j++; }
    }
    cout << best << '\n';
}

```

Complexity & pitfalls

Approach	Time	Memory
Brute force	$O(n^2)$	$O(n)$
Event sort	$O(n \log n)$	$O(n)$
Two sorted lists	$O(n \log n)$	$O(n)$

WATCH OUT

A “difference array” over all times (`cnt[a]++`, `cnt[b]--`) doesn’t work directly, because times reach 10^9 . It only works after coordinate compression, and at that point sorting the events is simpler.

6 • Movie Festival

cses.fi/problemset/task/1629 • Pattern: **interval scheduling: greedy by end time**

The task

n movies, each with a start and an end time. You can watch a movie only from start to end, and you can't watch two movies at once. Leaving one movie and starting another at the *same* moment is allowed. What is the **maximum number of movies** you can watch in full?

- $n \leq 2 \cdot 10^5$.

Building the intuition

Step 1: try some natural greedy rules and break them.

Rule	Counterexample
Shortest movie first	[1, 5] [4, 7] [6, 10] : the short one in the middle blocks both others
Earliest start first	[1, 100] [2, 3] [4, 5] : the early long movie blocks everything
Earliest end first	no counterexample exists, see below

Step 2: why “earliest end” is right. Think about the *first* movie in some optimal plan. Replace it with the movie that ends earliest overall. That movie ends no later, so it clashes with nothing the old first movie didn't clash with, and the rest of the plan still fits. So **some optimal plan starts with the earliest-ending movie**. Take it, discard everything that starts before it ends, and repeat the same argument on what's left.

KEY IDEA

Finishing early leaves the most time for everything else. “Pick the earliest-ending interval that still fits” is **the** classic interval scheduling greedy, and Movie Festival II (problem 34) extends it to k people.

WORKED EXAMPLE

Movies (3,5), (4,9), (5,8). Sorted by end: (3,5), (5,8), (4,9).

```
(3,5): free at 0, 3 >= 0    -> watch, free at 5    count = 1
(5,8): 5 >= 5              -> watch, free at 8    count = 2
(4,9): 4 < 8               -> skip                answer = 2
```

Approach 1 • Brute force, $O(2^n \cdot n \log n)$

Try every subset, sort it, and check that consecutive movies don't overlap. This is only good for checking tiny cases.

Approach 2 · Sort by end, greedy, $O(n \log n)$

We store each movie as (end, start) so that sorting the pairs sorts by end time automatically.

```
void solve() {
    int n; cin >> n;
    vector<pair<int,int>> mv(n);           // stored as (end, start)
    for (auto &[e, s] : mv) cin >> s >> e;
    sort(mv.begin(), mv.end());           // earliest ending first

    int count = 0, freeAt = 0;            // freeAt = when our last movie ends
    for (auto [e, s] : mv) {
        if (s >= freeAt) {                 // we can make it
            count++;
            freeAt = e;
        }
    }
    cout << count << '\n';
}
```

Approach 3 · Sort by start, “replace if it ends earlier”, $O(n \log n)$

The same greedy, seen from the other side. Go through movies by **start** time and keep the end time of the last movie in the plan.

- If the new movie starts after `lastEnd`, add it.
- If it clashes but **ends earlier** than the last planned movie, swap it in. The count stays the same, but we are now free earlier, which can only help.

```
void solve() {
    int n; cin >> n;
    vector<pair<int,int>> mv(n);           // (start, end)
    for (auto &[s, e] : mv) cin >> s >> e;
    sort(mv.begin(), mv.end());           // by start time

    int count = 0, lastEnd = 0;            // end of the last movie in our plan
    for (auto [s, e] : mv) {
        if (s >= lastEnd) {                 // no clash: add it
            count++;
            lastEnd = e;
        } else if (e < lastEnd) {           // clash, but this one ends earlier:
            lastEnd = e;                    // swap it in (count stays the same)
        }
    }
    cout << count << '\n';
}
```

Complexity & pitfalls

Approach	Time	Memory
Sort by end	$O(n \log n)$	$O(n)$
Sort by start + replace	$O(n \log n)$	$O(n)$

WATCH OUT

- Use `s >= freeAt` , not `>` : a movie that starts exactly when the previous one ends is allowed.
- Sorting `(start, end)` pairs and then greedily picking by start (without the replace step) is **wrong**.

7 • Sum of Two Values

cses.fi/problemset/task/1640 · Pattern: **complement lookup** / **two pointers**

The task

Given n integers and a target x , find **two different positions** whose values sum to x . Print the positions (1-based, any valid answer), or `IMPOSSIBLE`.

- $n \leq 2 \cdot 10^5$, x and $a_i \leq 10^9$.

Building the intuition

Step 1: fix one number and the other is determined. If one of the two numbers is v , the other *must* be $x - v$. So the question becomes: *for each element, does its complement $x - v$ exist at another position?*

Step 2: remember what you've seen. Walk from left to right. Before storing the current value, ask whether its complement appeared **earlier**. If it did, we have our pair. Checking *before* inserting also guarantees we never pair an element with itself.

Step 3, another view: sorted order gives a direction. If the array is sorted and we look at the smallest and largest remaining values:

- sum too small, so the smallest value can't be in any pair (even with the largest it's too small): drop it,
- sum too big, so the largest can't be in any pair: drop it,
- equal: found.

Each step discards one element, so this is $O(n)$ after sorting. We just have to remember the original positions.

WORKED EXAMPLE

$x = 8$, $a = 2\ 7\ 5\ 1$.

```
i=1 v=2 need 6 -> not seen, store 2
i=2 v=7 need 1 -> not seen, store 7
i=3 v=5 need 3 -> not seen, store 5
i=4 v=1 need 7 -> seen at position 2 -> answer "2 4"
```

Approach 1 • Brute force, $O(n^2)$

```
void solve() {
    int n, x; cin >> n >> x;
    vector<int> a(n);
    for (auto &v : a) cin >> v;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (a[i] + a[j] == x) { cout << i + 1 << " " << j + 1 << '\n'; return; }
    cout << "IMPOSSIBLE\n";
}
```

Approach 2 · Map of seen values, $O(n \log n)$

```
void solve() {
    int n, x; cin >> n >> x;
    map<int,int> seen;           // value -> a position where we saw it
    for (int i = 1; i <= n; i++) {
        int v; cin >> v;
        auto it = seen.find(x - v); // has the partner appeared before?
        if (it != seen.end()) {
            cout << it->second << " " << i << '\n';
            return;
        }
        seen[v] = i;             // insert AFTER checking -> never pairs i with itself
    }
    cout << "IMPOSSIBLE\n";
}
```

Approach 3 · Sort with indices + two pointers, $O(n \log n)$

```
void solve() {
    int n, x; cin >> n >> x;
    vector<pair<int,int>> a(n); // (value, original position)
    for (int i = 0; i < n; i++) { cin >> a[i].first; a[i].second = i + 1; }
    sort(a.begin(), a.end());

    int l = 0, r = n - 1;
    while (l < r) {
        int sum = a[l].first + a[r].first;
        if (sum == x) { cout << a[l].second << " " << a[r].second << '\n'; return; }
        if (sum < x) l++; // need bigger: move left pointer right
        else r--; // need smaller: move right pointer left
    }
    cout << "IMPOSSIBLE\n";
}
```

WHY IT IS CORRECT

Why can two pointers never skip the answer? Suppose the answer is the pair (p, q) in sorted order, with $p < q$. The pointers only move inward. If l passed p first, then at that moment $r \geq q$, so $a[l] + a[r] \geq a[p] + a[q] = x$. But we only move l when the sum is $< x$, a contradiction. The same argument works for r .

Complexity & pitfalls

Approach	Time	Memory
Brute force	$O(n^2)$	$O(1)$
Map	$O(n \log n)$	$O(n)$
Two pointers	$O(n \log n)$	$O(n)$

WATCH OUT

- $x = 4$ and a single value 2: storing *before* checking would wrongly answer “i i”.
- `unordered_map` with the default hash can be attacked on CSES. Use `map` or the `custom_hash` from problem 1.

8 • Maximum Subarray Sum

cses.fi/problemset/task/1643 • Pattern: **Kadane** / **prefix sums**

The task

Find the maximum sum of a **non-empty** contiguous subarray.

- $n \leq 2 \cdot 10^5$, values between -10^9 and 10^9 .

Building the intuition

Step 1: fix the right end. Every subarray ends somewhere. If we know, for every position r , the best sum of a subarray **ending exactly at r** , the answer is the largest of those.

Step 2: extend or restart. Let `cur` be the best sum ending at $r - 1$. A subarray ending at r either

- is just `a[r]` alone, or
- extends the best subarray ending at $r - 1$: `cur + a[r]`.

So `cur = max(a[r], cur + a[r])`. In words: **if the run so far has a negative sum, drop it and start fresh at $a[r]$** . That is Kadane's algorithm.

Step 3, another view: prefix sums. With $P[r] = a_1 + \dots + a_r$, the sum of $a[l+1..r]$ is $P[r] - P[l]$. For a fixed r we want the **smallest $P[l]$ with $l < r$** . We keep a running minimum.

WORKED EXAMPLE

`a = -1 3 -2 5 3 -5 2 2`

<code>a[r]</code>	:	-1	3	-2	5	3	-5	2	2	
<code>cur</code>	:	-1	3	1	6	9	4	6	8	
<code>best</code>	:	-1	3	3	6	9	9	9	9	<code>answer 9 (3 -2 5 3)</code>

Approach 1 • Brute force, $O(n^2)$

```
void solve() {
    int n; cin >> n;
    vector<int> a(n);
    for (auto &v : a) cin >> v;
    int best = LLONG_MIN;
    for (int l = 0; l < n; l++) {
        int sum = 0;
        for (int r = l; r < n; r++) { sum += a[r]; best = max(best, sum); }
    }
    cout << best << '\n';
}
```

Approach 2 • Kadane, $O(n)$

```
void solve() {
    int n; cin >> n;
    int best = LLONG_MIN, cur = 0; // cur = best sum of a subarray ENDING here
    for (int i = 0; i < n; i++) {
```

```

    int x; cin >> x;
    cur = max(x, cur + x);           // extend the previous run, or restart at x
    best = max(best, cur);
}
cout << best << '\n';
}

```

Approach 3 · Prefix sum minus minimum prefix, $O(n)$

```

void solve() {
    int n; cin >> n;
    int pref = 0, minPref = 0, best = LLONG_MIN; // minPref = min over P[0..r-1]
    for (int r = 1; r <= n; r++) {
        int x; cin >> x;
        pref += x; // pref = P[r]
        best = max(best, pref - minPref); // best subarray ending at r
        minPref = min(minPref, pref); // now P[r] may be used later
    }
    cout << best << '\n';
}

```

KEY IDEA

The two approaches are really the same thing: `cur` in Kadane equals $P[r] - \min(P[0..r-1])$. The prefix-sum form generalizes much better, though. Problem 35 (length between a and b) uses exactly this form with a *sliding* minimum.

Complexity & pitfalls

Approach	Time	Memory
Brute force	$O(n^2)$	$O(n)$
Kadane	$O(n)$	$O(1)$
Prefix sums	$O(n)$	$O(1)$

WATCH OUT

- **All negative input** (e.g. `-5 -2 -9`): the answer is -2 , not 0 . Start `best` at `LLONG_MIN`, not 0 , because the subarray must be non-empty.
- The sum can reach $2 \cdot 10^{14}$, beyond 32-bit. The template's `long long` handles it.

9 • Stick Lengths

cses.fi/problemset/task/1074 • Pattern: **median minimizes the sum of distances**

The task

n sticks with lengths p_i . You may lengthen or shorten any stick, and changing a length by d costs d . Make **all sticks equal** at minimum total cost.

- $n \leq 2 \cdot 10^5$, $p_i \leq 10^9$.

Building the intuition

Step 1: write down the cost. If every stick becomes length t , the cost is $f(t) = \sum |p_i - t|$: the total distance from t to all the points.

Step 2: move t and watch the cost. Sort the lengths. Suppose L sticks are $\leq t$ and R sticks are $> t$. Moving t one unit to the right makes each of the L sticks one unit farther ($+L$) and each of the R sticks one unit closer ($-R$). The change is $L - R$.

- While fewer than half of the sticks are on the left ($L < R$), moving right **decreases** the cost.
- Once more than half are on the left, moving right **increases** it.

Step 3: so the best t is where the two sides balance: the median. For odd n it's the middle element. For even n , any t between the two middle elements gives the same cost, so $p[n/2]$ always works.

WORKED EXAMPLE

$p = 2\ 3\ 1\ 5\ 2$, sorted $1\ 2\ 2\ 3\ 5$, median 2 .

$$\text{cost} = |1-2| + |2-2| + |2-2| + |3-2| + |5-2| = 1 + 0 + 0 + 1 + 3 = 5$$

Compare with $t = 3$ (the average rounded): cost $2+1+1+0+2 = 6$. **The mean is not the answer; the median is.**

Approach 1 • Brute force over targets

Try every t between the minimum and maximum. This works for tiny values only, but it shows that the cost curve is “V-shaped” (convex).

Approach 2 • Median, $O(n \log n)$

```
void solve() {
    int n; cin >> n;
    vector<int> p(n);
    for (auto &v : p) cin >> v;
    sort(p.begin(), p.end());
    int median = p[n / 2];           // any value between the two middles works
}
```

```

int cost = 0;
for (int v : p) cost += abs(v - median);
cout << cost << '\n';
}

```

`nth_element(p.begin(), p.begin() + n/2, p.end())` finds the median in $O(n)$ average without fully sorting.

Approach 3 · Try every stick length with prefix sums, $O(n \log n)$

If you don't want to rely on the median fact, compute $f(t)$ for **every** candidate $t = p[i]$ in $O(1)$. The optimum is always at one of the given lengths, because f is piecewise linear and only bends at the $p[i]$.

- sticks left of i are raised: cost $t \cdot i - (p[0] + \dots + p[i-1])$
- sticks right of i are cut: cost $(p[i+1] + \dots + p[n-1]) - t \cdot (n-1-i)$

```

// Evaluate the cost of EVERY candidate target p[i] in O(1) using prefix sums
void solve() {
    int n; cin >> n;
    vector<int> p(n);
    for (auto &v : p) cin >> v;
    sort(p.begin(), p.end());
    vector<int> pre(n + 1, 0);
    for (int i = 0; i < n; i++) pre[i + 1] = pre[i] + p[i];

    int best = LLONG_MAX;
    for (int i = 0; i < n; i++) {
        int t = p[i];
        int left = t * i - pre[i];
        int right = (pre[n] - pre[i + 1]) - t * (n - 1 - i); // raise p[0..i-1] up to t // cut p[i+1..n-1] down to t
        best = min(best, left + right);
    }
    cout << best << '\n';
}

```

KEY IDEA

This “evaluate every candidate using prefix sums” pattern is how you handle the variant where raising and lowering have **different** costs per unit. There the median argument changes, but the prefix-sum evaluation still works.

Complexity & pitfalls

Approach	Time	Memory
Median	$O(n \log n)$, or $O(n)$ with <code>nth_element</code>	$O(n)$
All candidates + prefix sums	$O(n \log n)$	$O(n)$

WATCH OUT

The total cost can reach $2 \cdot 10^5 \cdot 10^9 = 2 \cdot 10^{14}$, which needs 64-bit integers (handled by the template).

10 • Missing Coin Sum

cses.fi/problemset/task/2183 · Pattern: **sorted greedy with a reachability invariant**

The task

You have n coins with positive values. What is the **smallest positive sum you cannot make** using a subset of the coins?

- $n \leq 2 \cdot 10^5$, coin values $\leq 10^9$.

Building the intuition

Step 1: subset sums are hopeless directly. With values up to 10^9 , we can't list all reachable sums. We need a structural insight.

Step 2: suppose we can make every sum $1, 2, \dots, R$ (and $0 = \text{take nothing}$). What happens if we add a coin of value v ?

- Every old sum s stays makeable.
- Every $s + v$, for s in $[0, R]$, becomes makeable, i.e. every sum in $[v, v + R]$.

If $v \leq R + 1$, the new range $[v, v + R]$ touches or overlaps $[0, R]$, so now **everything in $[1, R + v]$ is makeable**.

If $v > R + 1$, the sum $R + 1$ is skipped. And if we add coins in **increasing** order, every later coin is $\geq v > R + 1$ as well, so $R + 1$ can never be formed. That's the answer.

Step 3: that's why we sort. Processing coins from small to large, the "reach" R grows as long as the next coin is $\leq R + 1$. The first gap gives the answer.

WORKED EXAMPLE

Coins 2 9 1 2 7, sorted 1 2 2 7 9.

```
R = 0
coin 1 : 1 <= 1 -> R = 1      (can make 1)
coin 2 : 2 <= 2 -> R = 3      (1..3)
coin 2 : 2 <= 4 -> R = 5      (1..5)
coin 7 : 7 > 6 -> STOP. 6 is impossible.      answer 6
```

Approach 1 • Brute force (subset sums)

Build the set of all subset sums and find the first missing positive. This is exponential in the worst case, but perfect for checking small inputs.

```
void solve() {
    int n; cin >> n;
    vector<int> c(n);
    for (auto &v : c) cin >> v;
    set<int> sums = {0};
    for (int v : c) {
        vector<int> add;
        for (int s : sums) add.push_back(s + v);
        // classic subset-sum set
    }
}
```

```

        for (int s : add) sums.insert(s);
    }
    int ans = 1;
    while (sums.count(ans)) ans++;
    cout << ans << '\n';
}

```

Approach 2 · Sort + running reach, $O(n \log n)$

```

void solve() {
    int n; cin >> n;
    vector<int> c(n);
    for (auto &v : c) cin >> v;
    sort(c.begin(), c.end());

    int reach = 0; // invariant: every sum in [1, reach] is makeable
    for (int v : c) {
        if (v > reach + 1) break; // reach+1 can never be formed
        reach += v; // now every sum in [1, reach + v] is makeable
    }
    cout << reach + 1 << '\n';
}

```

WHY IT IS CORRECT

Invariant: after processing coins $c_1 \leq \dots \leq c_i$, every value in $[1, R]$ is makeable and $R + 1$ is not makeable *from these coins*. Adding $c_{i+1} \leq R + 1$ extends the range to $[1, R + c_{i+1}]$ as shown in Step 2. If $c_{i+1} > R + 1$, then every remaining coin is also $> R + 1$, so no subset containing any of them can sum to exactly $R + 1$. The coins already used can't make it either.

Complexity & pitfalls

Approach	Time	Memory
Subset sums	exponential	exponential
Sort + reach	$O(n \log n)$	$O(n)$

WATCH OUT

- The answer can be as large as $2 \cdot 10^{14} + 1$, so use 64-bit integers.
- If there is no coin of value 1, the answer is 1. No special case is needed, because the very first check `c > 0 + 1` fires for any coin ≥ 2 .

11 • Collecting Numbers

cses.fi/problemset/task/2216 · Pattern: **think in positions, not values**

The task

The array is a permutation of $1 \dots n$. You collect the numbers **in increasing order** $1, 2, \dots, n$. In each round you walk the array once from left to right and pick up as many numbers as you can in order. How many rounds are needed?

- $n \leq 2 \cdot 10^5$.

Building the intuition

Step 1: simulate a small case. Array `4 2 1 5 3`:

```
round 1: walk left->right, need 1: skip 4, skip 2, take 1, need 2... 2 is behind us. stop.
round 2: take 2, then 3 (at the end).          need 4 ... behind us. stop.
round 3: take 4, take 5.                      done: 3 rounds
```

Step 2: when does a new round start? We finish collecting x and need $x + 1$. If $x + 1$ is **to the right** of x , we keep walking and grab it in the same round. If $x + 1$ is **to the left** of x , we've already passed it, so a new round is needed.

Step 3: so only the relative order of x and $x + 1$ matters. Let $\text{pos}[v]$ be the index of value v . Then

$\text{rounds} = 1 + (\text{number of } x \text{ in } 1 \dots n-1 \text{ with } \text{pos}[x+1] < \text{pos}[x]).$

KEY IDEA

Switching from “value at each position” ($a[i]$) to “position of each value” ($\text{pos}[v]$) is the whole problem. Whenever a problem talks about values in **sorted order**, try storing $\text{pos}[]$.

WORKED EXAMPLE

`a = 4 2 1 5 3` $\rightarrow$ $\text{pos}[1]=2, \text{pos}[2]=1, \text{pos}[3]=4, \text{pos}[4]=0, \text{pos}[5]=3$ (0-based)

```
pair (1,2): pos 1 < 2    -> 2 is behind 1 -> +1
pair (2,3): pos 4 > 1    -> fine
pair (3,4): pos 0 < 4    -> +1
pair (4,5): pos 3 > 0    -> fine          rounds = 1 + 2 = 3
```

Approach 1 • Simulate the rounds, $O(n \cdot \text{rounds})$

Straightforward, but a reversed array needs n rounds, which is $O(n^2)$.

```
// literally simulate the rounds
void solve() {
    int n; cin >> n;
    vector<int> a(n);
    for (auto &v : a) cin >> v;
    int next = 1, rounds = 0;
    while (next <= n) {
        rounds++;
    }
```

```

        for (int i = 0; i < n; i++) if (a[i] == next) next++;
    }
    cout << rounds << '\n';
}

```

Approach 2 · Count “broken” neighbouring pairs, $O(n)$

```

void solve() {
    int n; cin >> n;
    vector<int> pos(n + 1);
    for (int i = 0; i < n; i++) { int x; cin >> x; pos[x] = i; }

    int rounds = 1; // we always need at least one round
    for (int x = 1; x < n; x++)
        if (pos[x + 1] < pos[x]) rounds++; // x+1 is behind x -> must wait for a new round
    cout << rounds << '\n';
}

```

Approach 3 · One left-to-right scan, $O(n)$

The same rule seen while reading the array: when we meet x , has $x - 1$ already appeared? If not, $x - 1$ is to the right, so x cannot be taken in the same round as $x - 1$, and **x starts a new round**. ($x = 1$ always starts round 1.)

```

void solve() {
    int n; cin >> n;
    vector<bool> seen(n + 1, false);
    seen[0] = false; // value 0 doesn't exist -> "1" always opens a round
    int rounds = 0;
    for (int i = 0; i < n; i++) {
        int x; cin >> x;
        if (!seen[x - 1]) rounds++; // x-1 hasn't appeared yet, so x cannot be picked
        seen[x] = true; // in the same pass as x-1 -> x starts a new round
    }
    cout << rounds << '\n';
}

```

Complexity & pitfalls

Approach	Time	Memory
Simulation	$O(n^2)$ worst case	$O(n)$
Pairs via <code>pos[]</code>	$O(n)$	$O(n)$
One scan	$O(n)$	$O(n)$

12 • Collecting Numbers II

cses.fi/problemset/task/2217 · Pattern: **maintain an answer under local updates**

The task

Same game as problem 11, but now there are m operations. Each one **swaps the elements at positions a and b** , and after every swap you must print the number of rounds.

- $n, m \leq 2 \cdot 10^5$, so recomputing $O(n)$ per swap ($4 \cdot 10^{10}$ total) is too slow.

Building the intuition

Step 1: reuse the formula. From problem 11: $\text{rounds} = 1 + \sum \text{bad}(v)$, where pair v means the values $(v, v + 1)$ and $\text{bad}(v) = 1$ if $\text{pos}[v+1] < \text{pos}[v]$.

Step 2: what does a swap actually change? Swapping positions x and y moves exactly **two values**, $u = a[x]$ and $w = a[y]$. Only $\text{pos}[u]$ and $\text{pos}[w]$ change. A pair $(v, v + 1)$ can only change its status if it **contains u or w** . That's at most 4 pairs:

$(u - 1, u)$, $(u, u + 1)$, $(w - 1, w)$, $(w, w + 1)$.

Step 3: update in three moves.

1. Subtract the current contribution of those pairs.
2. Perform the swap and update `pos`.
3. Add the new contribution of the same pairs.

That's $O(1)$ work per swap.

Step 4: the trap, overlapping pairs. If u and w are neighbours, say $w = u + 1$, then the pair $(u, u + 1)$ is **the same pair** as $(w - 1, w)$. If we subtract it twice and add it twice, we get the wrong total. We need to deduplicate the pairs, and a tiny `set` does the job.

WORKED EXAMPLE

$a = 4\ 2\ 1\ 5\ 3$ (3 rounds). Swap positions 2 and 3 $\rightarrow a = 4\ 1\ 2\ 5\ 3$. The moved values are $u = 2$ and $w = 1$.

```
pairs to recheck: {u-1, u, w-1, w} = {1, 2, 0, 1} -> {0, 1, 2} (pair 0 doesn't exist)
before: pair1 (1,2) bad, pair2 (2,3) fine -> rounds -= 1 -> 2
after : pair1 (1,2) fine, pair2 (2,3) fine -> rounds += 0 -> 2      print 2
```

Approach 1 • Recompute after every swap, $O(n \cdot m)$

```
void solve() {
    int n, m; cin >> n >> m;
    vector<int> a(n + 1), pos(n + 2);
    for (int i = 1; i <= n; i++) { cin >> a[i]; pos[a[i]] = i; }
    while (m--) {
        int x, y; cin >> x >> y;
        swap(a[x], a[y]); pos[a[x]] = x; pos[a[y]] = y;
        int rounds = 1;
    }
}
```

```

    for (int v = 1; v < n; v++) if (pos[v + 1] < pos[v]) rounds++;
    cout << rounds << '\n';
}
}

```

Approach 2 · Remove, swap, add back, $O(n + m)$

```

int n, m;
vector<int> a, pos;

// pair (v, v+1) costs an extra round if v+1 sits before v
int bad(int v) {
    if (v < 1 || v >= n) return 0;          // pair doesn't exist
    return pos[v + 1] < pos[v];
}

void solve() {
    cin >> n >> m;
    a.assign(n + 1, 0); pos.assign(n + 2, 0);
    for (int i = 1; i <= n; i++) { cin >> a[i]; pos[a[i]] = i; }

    int rounds = 1;
    for (int v = 1; v < n; v++) rounds += bad(v);

    while (m--) {
        int x, y; cin >> x >> y;
        int u = a[x], w = a[y];              // the two VALUES being moved
        // only pairs that contain u or w can change
        set<int> pairs = {u - 1, u, w - 1, w}; // pair v means (v, v+1); set removes
        duplicates

        for (int v : pairs) rounds -= bad(v); // 1) remove old contribution
        swap(a[x], a[y]);                     // 2) do the swap
        pos[a[x]] = x; pos[a[y]] = y;
        for (int v : pairs) rounds += bad(v); // 3) add new contribution

        cout << rounds << '\n';
    }
}

```

Approach 3 · Keep a `broken[]` flag per pair, $O(n + m)$

Instead of “subtract then add”, store the flag of every pair. `refresh(v)` recomputes one pair’s flag and fixes the total by the difference. Because refreshing the same pair twice is harmless (the second time the difference is 0), **no deduplication is needed**. This is a nice way to make updates robust.

```

// Same idea, but store the "is broken" flag of every pair explicitly.
// Recomputing a flag is idempotent, so duplicates need no special care.
void solve() {
    int n, m; cin >> n >> m;
    vector<int> a(n + 1), pos(n + 2);
    for (int i = 1; i <= n; i++) { cin >> a[i]; pos[a[i]] = i; }

    vector<int> broken(n + 1, 0); // broken[v] = 1 if pos[v+1] < pos[v]
    int rounds = 1;
    auto refresh = [&](int v) {
        if (v < 1 || v >= n) return;
        rounds -= broken[v];
        broken[v] = pos[v + 1] < pos[v];
        rounds += broken[v];
    };
}

```

```

for (int v = 1; v < n; v++) refresh(v);

while (m--) {
    int x, y; cin >> x >> y;
    int u = a[x], w = a[y];
    swap(a[x], a[y]);
    pos[u] = y; pos[w] = x;
    refresh(u - 1); refresh(u); refresh(w - 1); refresh(w);
    cout << rounds << '\n';
}

```

KEY IDEA

The general technique: **find the few pieces of the answer that an update can touch**, remove their old contribution, apply the update, and add their new contribution. It shows up in many problems with point updates.

Complexity & pitfalls

Approach	Time	Memory
Recompute	$O(n \cdot m)$	$O(n)$
Remove / add with set	$O(n + m)$	$O(n)$
<code>broken[]</code> + refresh	$O(n + m)$	$O(n)$

WATCH OUT

- Adjacent values ($|u - w| = 1$) share a pair: deduplicate, or use the idempotent `refresh`.
- Save $u = a[x]$ and $w = a[y]$ **before** swapping.
- The input may swap a position with itself ($x = y$); both solutions handle it.

13 · Playlist

cses.fi/problemset/task/1141 · Pattern: **sliding window with distinct elements**

The task

Given a sequence of n songs (by ID), find the **longest contiguous block with no repeated song**.

- $n \leq 2 \cdot 10^5$, IDs up to 10^9 .

Building the intuition

Step 1: fix the right end r . Among windows ending at r , which is the longest valid one? It starts at the smallest l such that $k[l..r]$ has no duplicates. Call it $L(r)$.

Step 2: L never moves left. If $k[l..r]$ has a duplicate, then so does $k[l..r+1]$, because adding an element can't remove a duplicate. So as r grows, $L(r)$ can only stay or move right.

Step 3: that is exactly a two-pointer / sliding window. Move r forward one step at a time. When the new song is already in the window, move l forward until it isn't. Each index enters and leaves the window at most once, which is $O(n)$ steps in total.

WORKED EXAMPLE

Songs 1 2 1 3 2 7 4 2

$r=0$	[1]	len	1	
$r=1$	[1 2]	len	2	
$r=2$	1 repeats -> drop 1	[2 1]	len	2
$r=3$	[2 1 3]	len	3	
$r=4$	2 repeats -> drop 2	[1 3 2]	len	3
$r=5$	[1 3 2 7]	len	4	
$r=6$	[1 3 2 7 4]	len	5	<- best
$r=7$	2 repeats -> drop 1,3,2	[7 4 2]	len	3
				answer 5

Approach 1 · Brute force, $O(n^2 \log n)$

From every start, extend while no song repeats.

```
void solve() {
    int n; cin >> n;
    vector<int> k(n);
    for (auto &v : k) cin >> v;
    int best = 0;
    for (int l = 0; l < n; l++) {
        set<int> s;
        for (int r = l; r < n && !s.count(k[r]); r++) { s.insert(k[r]); best = max(best, r - l + 1); }
    }
    cout << best << '\n';
}
```

Approach 2 · Window with a set, $O(n \log n)$

```
void solve() {
    int n; cin >> n;
    vector<int> k(n);
    for (auto &v : k) cin >> v;

    set<int> window;           // songs currently inside [l, r]
    int l = 0, best = 0;
    for (int r = 0; r < n; r++) {
        while (window.count(k[r])) { // k[r] would repeat -> shrink from the left
            window.erase(k[l]);
            l++;
        }
        window.insert(k[r]);
        best = max(best, r - l + 1);
    }
    cout << best << '\n';
}
```

Approach 3 · Jump with “last seen” positions, $O(n \log n)$

Instead of removing songs one by one, remember where each song was **last** seen. When song x appears again, the window must start right after its previous copy: $l = \max(l, \text{last}[x] + 1)$. The `max` matters: an old copy that’s already outside the window must not pull `l` back to the left.

```
void solve() {
    int n; cin >> n;
    map<int,int> last;           // song -> last index where it appeared
    int l = 0, best = 0;
    for (int r = 0; r < n; r++) {
        int x; cin >> x;
        auto it = last.find(x);
        if (it != last.end()) l = max(l, it->second + 1); // jump l past the old copy
        last[x] = r;
        best = max(best, r - l + 1);
    }
    cout << best << '\n';
}
```

Complexity & pitfalls

Approach	Time	Memory
Brute force	$O(n^2 \log n)$	$O(n)$
Set window	$O(n \log n)$	$O(n)$
Last-seen jump	$O(n \log n)$ ($O(n)$ with compression)	$O(n)$

WATCH OUT

Forgetting the `max` in $l = \max(l, \text{last}[x] + 1)$: in `1 2 2 1`, the last 1 would move `l` back to 1 and report 3 instead of 2.

14 • Towers

cses.fi/problemset/task/1073 · Pattern: **greedy with upper_bound (patience sorting)**

The task

Cubes arrive one by one with sizes $k_1, \dots, k_n$. Each cube is either placed on top of an existing tower, which requires the tower's top cube to be **strictly larger**, or starts a new tower. Minimize the number of towers.

- $n \leq 2 \cdot 10^5$.

Building the intuition

Step 1: only the tops matter. A tower's history is irrelevant; the only thing that decides where future cubes can go is its **top cube**. So the state is just the multiset of tops.

Step 2: a cube k can go on any tower with top $> k$. Which one should it choose? Placing k on a tower replaces its top t with the smaller k , which makes that tower "less welcoming" for future cubes. To do the least damage, sacrifice the tower with the **smallest top that is still $> k$** . The towers with larger tops stay available for larger future cubes.

Step 3: if no top is $> k$, a new tower is unavoidable.

In multiset terms: `it = tops.upper_bound(k)` (the first top strictly greater than k). If it exists, replace it with k ; otherwise insert k .

WORKED EXAMPLE

Cubes 3 8 2 1 5

3	: no top > 3	-> new tower	tops {3}	
8	: no top > 8	-> new tower	tops {3, 8}	
2	: smallest top > 2 is 3	-> replace	tops {2, 8}	
1	: smallest top > 1 is 2	-> replace	tops {1, 8}	
5	: smallest top > 5 is 8	-> replace	tops {1, 5}	answer 2

WHY IT IS CORRECT

Exchange argument. Suppose an optimal strategy puts k on a tower with top $t' > t$, where t is the smallest top $> k$. Swap the futures of these two towers: everything the optimal strategy later stacks on the t' -tower goes on the t -tower instead, and vice versa. After placing k , the towers have tops $\{k, t'\}$ in our version and $\{t, k\}$ in theirs. Our version's tops are elementwise at least as large, so every later placement remains legal and the tower count doesn't change.

Step 4, a surprise: the tops always stay **sorted in the order they were created**, so a plain vector with `upper_bound` works, without any multiset. This is exactly *patience sorting*, and the number of towers equals the length of the **longest non-decreasing subsequence** of the input (the cubes of a non-decreasing subsequence must all end up in different towers, and the greedy achieves exactly that bound). The brute-force checker below uses this fact.

Approach 1 · Brute force via the LNDS connection, $O(n^2)$

```
// min #towers = length of the longest NON-DECREASING subsequence ( $O(n^2)$  DP)
void solve() {
    int n; cin >> n;
    vector<int> k(n);
    for (auto &v : k) cin >> v;
    vector<int> dp(n, 1);
    int best = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) if (k[j] <= k[i]) dp[i] = max(dp[i], dp[j] + 1);
        best = max(best, dp[i]);
    }
    cout << best << '\n';
}
```

Approach 2 · Multiset of tops, $O(n \log n)$

```
void solve() {
    int n; cin >> n;
    multiset<int> tops; // top cube of every tower
    for (int i = 0; i < n; i++) {
        int k; cin >> k;
        auto it = tops.upper_bound(k); // smallest top that is STRICTLY larger than k
        if (it != tops.end()) tops.erase(it); // put cube on that tower: its old top disappears
        tops.insert(k); // ... and k becomes a top (new tower or old one)
    }
    cout << tops.size() << '\n';
}
```

Approach 3 · Sorted vector of tops, $O(n \log n)$

Replacing `*it` with `k` keeps the vector sorted: the element before `it` is $\leq k$ (that's what `upper_bound` guarantees), and the old `*it` was $> k$.

```
// The tops, if kept in a vector, are ALWAYS sorted -> binary search on a plain vector
void solve() {
    int n; cin >> n;
    vector<int> tops; // sorted ascending at all times
    for (int i = 0; i < n; i++) {
        int k; cin >> k;
        auto it = upper_bound(tops.begin(), tops.end(), k);
        if (it == tops.end()) tops.push_back(k); // no tower can take it: new tower
        else *it = k; // replace that top; order stays sorted
    }
    cout << tops.size() << '\n';
}
```

Complexity & pitfalls

Approach	Time	Memory
LNDS DP	$O(n^2)$	$O(n)$
Multiset	$O(n \log n)$	$O(n)$
Vector (patience)	$O(n \log n)$, faster constant	$O(n)$

WATCH OUT

“Strictly larger” means `upper_bound`, not `lower_bound`. Two equal cubes can’t be stacked, so with `lower_bound` the input `2 2` would give 1 tower instead of 2.

15 • Traffic Lights

cses.fi/problemset/task/1163 · Pattern: **set of cut points + multiset of lengths / offline reverse**

The task

A street runs from 0 to x . Traffic lights are added one at a time at distinct positions. After each addition, print the **length of the longest segment without traffic lights**.

- $x \leq 10^9, n \leq 2 \cdot 10^5$.

Building the intuition

Step 1: what does one new light do? A light at p lands inside exactly one current segment $[L, R]$, where L is the nearest light (or 0) to the left and R the nearest to the right. That segment disappears and is replaced by $[L, p]$ and $[p, R]$. Nothing else changes.

Step 2: we need two structures.

- A sorted set of light positions (with 0 and x as sentinels) to find L and R : `upper_bound(p)` gives R , and the element before it is L .
- A **multiset of segment lengths** supporting “remove one $R - L$, add $p - L$ and $R - p$, ask for the maximum”.

Step 3, another view: run time backwards. Adding lights only *splits* segments, so the maximum can go down, and tracking a decreasing maximum needs a multiset. Removing lights only *merges* segments, so the maximum can only **go up**, and a single variable is enough. So: place all lights first, compute the answer for the final state, then remove the lights in reverse order and record the answers backwards. A doubly linked list over the sorted positions gives each light's neighbours in $O(1)$.

WORKED EXAMPLE

$x = 8$, lights 3, 6, 2

add 3 : [0,8] -> [0,3] [3,8]	lengths {3,5}	max 5	
add 6 : [3,8] -> [3,6] [6,8]	lengths {3,3,2}	max 3	
add 2 : [0,3] -> [0,2] [2,3]	lengths {2,1,3,2}	max 3	output: 5 3 3

Approach 1 • Brute force, $O(n^2 \log n)$

Re-sort all positions after each light and scan the gaps.

Approach 2 • **set + multiset**, $O(n \log n)$

```
void solve() {
    int x, n; cin >> x >> n;
    set<int> lights = {0, x};           // borders act like lights
    multiset<int> gaps = {x};           // lengths of all current segments

    for (int i = 0; i < n; i++) {
        int p; cin >> p;
        auto right = lights.upper_bound(p); // first light after p
```

```

    auto left = prev(right); // last light before p
    int L = *left, R = *right;

    gaps.erase(gaps.find(R - L)); // segment [L,R] is split... (erase ONE copy)
    gaps.insert(p - L); // ... into [L,p]
    gaps.insert(R - p); // ... and [p,R]
    lights.insert(p);

    cout << *gaps.rbegin() << " \n"[i == n - 1]; // longest segment
}
}

```

Approach 3 · Offline, in reverse, with a linked list, $O(n \log n)$

```

// Offline, backwards: start with ALL lights, then remove them in reverse order.
// Removing only MERGES segments, so the answer can only grow -> a single running max.
void solve() {
    int x, n; cin >> x >> n;
    vector<int> p(n);
    for (auto &v : p) cin >> v;

    vector<int> s = p;
    s.push_back(0); s.push_back(x);
    sort(s.begin(), s.end());
    int m = s.size();
    vector<int> L(m), R(m); // doubly linked list over sorted positions
    int best = 0;
    for (int i = 0; i < m; i++) {
        L[i] = i - 1; R[i] = i + 1;
        if (i) best = max(best, s[i] - s[i - 1]);
    }

    vector<int> ans(n);
    for (int i = n - 1; i >= 0; i--) {
        ans[i] = best; // answer after lights p[0..i] are placed
        int id = lower_bound(s.begin(), s.end(), p[i]) - s.begin();
        int a = L[id], b = R[id]; // neighbours of the light we remove
        R[a] = b; L[b] = a; // unlink it
        best = max(best, s[b] - s[a]); // the merged segment
    }
    for (int i = 0; i < n; i++) cout << ans[i] << " \n"[i == n - 1];
}

```

KEY IDEA

“Deleting is hard but inserting is easy” (or the other way round) is a strong hint to **reverse time**. The same trick powers offline solutions to many connectivity problems (removing edges = adding them in reverse with a DSU).

Complexity & pitfalls

Approach	Time	Memory
Brute force	$O(n^2 \log n)$	$O(n)$
Set + multiset	$O(n \log n)$	$O(n)$
Offline reverse	$O(n \log n)$ (just the sort), lighter constant	$O(n)$

WATCH OUT

`gaps.erase(R - L)` would erase **every** segment of that length. Use `gaps.erase(gaps.find(R - L))`.

16 · Distinct Values Subarrays

cses.fi/problemset/task/3420 · Pattern: **two pointers: count windows**

The task

Count the subarrays in which **all values are distinct**.

- $n \leq 2 \cdot 10^5$, values up to 10^9 . The answer can reach $n(n+1)/2 \approx 2 \cdot 10^{10}$, so it needs 64 bits.

Building the intuition

Step 1: this is the counting version of Playlist (problem 13). There we wanted the *longest* duplicate-free window. Now we want *how many* there are.

Step 2: good windows are closed under shrinking. If `a[l..r]` has no duplicates, then `a[l+1..r]` has none either. So for a fixed right end `r`, the good left ends form one contiguous block `[L(r), r]`, where `L(r)` is the smallest valid start.

Step 3: count per right end. For each `r`, exactly $r - L(r) + 1$ subarrays end at `r` and are good. Sum that over all `r`.

Step 4: L(r) never moves left (same argument as in Playlist), so a sliding window computes all `L(r)` in $O(n)$ pointer moves in total.

WORKED EXAMPLE

`a = 1 2 1 3`

```
r=0 : window [1]      L=0  -> 1 subarray  ([1])
r=1 : window [1 2]    L=0  -> 2          ([1 2], [2])
r=2 : 1 repeats, L=1   [2 1] -> 2        ([2 1], [1])
r=3 : window [2 1 3]  L=1  -> 3          ([2 1 3], [1 3], [3])
total = 1 + 2 + 2 + 3 = 8
```

KEY IDEA

“Count subarrays with property *P*” where *P* survives shrinking: for each right end, count the valid left ends as $r - l + 1$. The same template solves problems 29 and 32.

Approach 1 · Brute force, $O(n^2 \log n)$

```
void solve() {
    int n; cin >> n;
    vector<int> a(n);
    for (auto &v : a) cin >> v;
    int cnt = 0;
    for (int l = 0; l < n; l++) {
        set<int> s;
        for (int r = l; r < n && !s.count(a[r]); r++) { s.insert(a[r]); cnt++; }
    }
    cout << cnt << '\n';
}
```

Approach 2 · Sliding window with frequency map, $O(n \log n)$

```
void solve() {
    int n; cin >> n;
    vector<int> a(n);
    for (auto &v : a) cin >> v;

    map<int,int> freq;           // counts inside the window [l, r]
    int l = 0, total = 0;
    for (int r = 0; r < n; r++) {
        freq[a[r]]++;
        while (freq[a[r]] > 1) { // a[r] is duplicated -> shrink from the left
            freq[a[l]]--;
            l++;
        }
        total += r - l + 1;      // [l..r], [l+1..r], ..., [r..r] are all good
    }
    cout << total << '\n';
}
```

Approach 3 · Compression + last occurrence, $O(n \log n)$ for the sort, then $O(n)$

Compress values to $0..n-1$ so a plain `vector` can store the last position of each value. The window's left end is then $\max(l, \text{last}[a[r]] + 1)$, exactly as in Playlist.

```
// Coordinate-compress values to 0..n-1, then use a plain array for "last position"
void solve() {
    int n; cin >> n;
    vector<int> a(n);
    for (auto &v : a) cin >> v;
    vector<int> vals = a;
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    for (auto &v : a) v = lower_bound(vals.begin(), vals.end(), v) - vals.begin();

    vector<int> last(vals.size(), -1);
    int l = 0, total = 0;
    for (int r = 0; r < n; r++) {
        l = max(l, last[a[r]] + 1); // window must start after the previous copy of a[r]
        last[a[r]] = r;
        total += r - l + 1;
    }
    cout << total << '\n';
}
```

Complexity & pitfalls

Approach	Time	Memory
Brute force	$O(n^2 \log n)$	$O(n)$
Window + map	$O(n \log n)$	$O(n)$
Compression + last	$O(n \log n)$	$O(n)$

WATCH OUT

The answer overflows 32-bit integers for large n (all distinct $\rightarrow n(n+1)/2$).

17 • Distinct Values Subsequences

cses.fi/problemset/task/3421 • Pattern: **counting with independent choices (product rule)**

The task

Count the non-empty **subsequences** (chosen by positions, not necessarily contiguous) in which **all values are distinct**. Print the answer modulo $10^9 + 7$.

- $n \leq 2 \cdot 10^5$.

Building the intuition

Step 1: order doesn't matter here. A subsequence keeps the original order automatically, so it is really just a **set of chosen positions**. The only constraint is: never choose two positions holding the same value.

Step 2: split the choice by value. Group the positions by value. Say value v appears $f(v)$ times. For this value we can:

- pick none of its copies: 1 way, or
- pick exactly one of its $f(v)$ copies: $f(v)$ ways.

That's $f(v) + 1$ options, and the choice for one value doesn't affect the choice for any other value.

Step 3: the product rule. Independent choices multiply:

$\text{answer} = \prod (f(v) + 1) - 1$, where the -1 removes the empty subsequence (choosing "none" for every value).

WORKED EXAMPLE

$a = 1\ 2\ 1\ 3$: $f(1) = 2$, $f(2) = 1$, $f(3) = 1$

$$(2+1) \cdot (1+1) \cdot (1+1) - 1 = 3 \cdot 2 \cdot 2 - 1 = 11$$

Approach 1 • Brute force over all 2^n subsets

Check every subset of positions for repeated values. It only works for $n \approx 20$, but it confirms the formula.

Approach 2 • Sort, count blocks, multiply, $O(n \log n)$

```
void solve() {
    int n; cin >> n;
    vector<int> a(n);
    for (auto &v : a) cin >> v;
    sort(a.begin(), a.end());           // equal values form one block

    int ways = 1;
    for (int i = 0; i < n; ) {
        int j = i;
        while (j < n && a[j] == a[i]) j++; // block [i, j) has f = j - i copies
        ways = ways * (j - i + 1) % mod;   // skip the value, or pick one of its f copies
        i = j;
    }
}
```

```

        i = j;
    }
    cout << (ways - 1 + mod) % mod << '\n'; // remove the empty subsequence
}

```

Approach 3 · Online with modular inverses, $O(n \log n)$

What if elements arrive one at a time and we must know the answer after each one? When a new copy of v arrives, its factor changes from $(c + 1)$ to $(c + 2)$. We **divide out** the old factor and multiply in the new one. Division modulo a prime p is multiplication by the inverse, $b^{p-2} \bmod p$ (Fermat's little theorem). This is overkill here, but it's a useful pattern when a product must be kept up to date.

```

// Online version: add elements one by one.
// ways(value v) goes from f to f+1 when a new copy arrives, so
// total = total / (f+1) * (f+2) -> needs a modular inverse, which is fine since mod is prime.
int power(int b, int e) {
    int r = 1; b %= mod;
    while (e) { if (e & 1) r = r * b % mod; b = b * b % mod; e >>= 1; }
    return r;
}

void solve() {
    int n; cin >> n;
    map<int,int> f;
    int ways = 1;
    for (int i = 0; i < n; i++) {
        int x; cin >> x;
        int c = f[x]++;
        ways = ways * power(c + 1, mod - 2) % mod; // copies before this one
        ways = ways * (c + 2) % mod; // remove old factor (c+1)
        // multiply new factor (c+2)
    }
    cout << (ways - 1 + mod) % mod << '\n';
}

```

Complexity & pitfalls

Approach	Time	Memory
Brute force	$O(2^n \cdot n)$	$O(n)$
Sort + product	$O(n \log n)$	$O(n)$
Online inverses	$O(n \log \text{mod})$	$O(n)$

WATCH OUT

- Take the product **mod 10^9+7 at every step**. The raw product is astronomically large.
- `(ways - 1 + mod) % mod`: if the product happens to be $\equiv 0 \pmod{p}$, plain `ways - 1` would be -1 .
- The modular-inverse trick is only safe because every factor $(c + 1) \leq n + 1$ is smaller than the prime, so it is never divisible by the prime.

18 • Josephus Problem I

cses.fi/problemset/task/2162 · Pattern: **simulation with a queue**

The task

Children $1 \dots n$ stand in a circle. Starting from child 1, **every second child** is removed: 1 is skipped, 2 is removed, 3 is skipped, 4 is removed, and so on around the circle. Print the removal order.

- $n \leq 2 \cdot 10^5$.

Building the intuition

Step 1: what does the circle need to support? “Skip one child” and “remove the next one”. Removing from the middle of a vector is $O(n)$, which is $O(n^2)$ overall. We want both steps to be $O(1)$.

Step 2: a queue is a circle. Put the children in a queue. “Skipping” the front child means moving it to the back (`push(front); pop()`). “Removing” means printing the front and popping it. The order of the queue always matches the order around the circle, starting from the current position.

WORKED EXAMPLE

$n = 7$

queue	action
1 2 3 4 5 6 7	skip 1, remove 2
3 4 5 6 7 1	skip 3, remove 4
5 6 7 1 3	skip 5, remove 6
7 1 3 5	skip 7, remove 1
3 5 7	skip 3, remove 5
7 3	skip 7, remove 3
7	skip 7, remove 7

output: 2 4 6 1 5 3 7

Approach 1 • Vector with `erase`, $O(n^2)$

Keep an index into a vector and erase at `(idx + 1) % size`. It works, but each erase shifts elements.

Approach 2 • Queue, $O(n)$

```
void solve() {
    int n; cin >> n;
    queue<int> q;
    for (int i = 1; i <= n; i++) q.push(i);

    while (!q.empty()) {
        q.push(q.front()); q.pop();    // skip one child: move him to the back
        cout << q.front() << ' ';    // the next child is removed
        q.pop();
    }
    cout << '\n';
}
```

Approach 3 · Round by round, $O(n)$

Walk over the survivors of each round and alternate “skip, remove, skip, remove...”. The alternation must **carry over** from one round to the next: if a round ends with a skip, the first child of the next round is removed. Each round halves the circle, so the total work is $n + n/2 + n/4 + \dots < 2n$.

```
// Round by round: walk over the survivors and remove every second one.
// "skip" carries over between rounds so the alternation never breaks.
void solve() {
    int n; cin >> n;
    vector<int> alive(n);
    iota(alive.begin(), alive.end(), 1);

    bool removeNext = false; // the first child (1) is skipped
    while (!alive.empty()) {
        vector<int> survivors;
        for (int c : alive) {
            if (removeNext) cout << c << ' ';
            else survivors.push_back(c);
            removeNext = !removeNext; // strictly alternate: skip, remove, skip, ...
        }
        alive = survivors; // each round halves the circle ->  $O(n)$  total
    }
    cout << '\n';
}
```

NOTE

There is also a closed form for the *last* survivor when every second child is removed (write n in binary, move the leading 1 to the end). That only gives the winner, though, not the whole order.

Complexity

Approach	Time	Memory
Vector erase	$O(n^2)$	$O(n)$
Queue	$O(n)$	$O(n)$
Rounds	$O(n)$	$O(n)$

19 • Josephus Problem II

cses.fi/problemset/task/2163 · Pattern: **order statistics**: “find the i -th alive element”

The task

Same circle of n children, but now in each step **k children are skipped** and the next one is removed. Print the removal order.

- $n \leq 2 \cdot 10^5$, $0 \leq k \leq 10^9$.

Building the intuition

Step 1: the queue from problem 18 is too slow now. Skipping k children one by one costs $O(k)$ per removal, and k can be 10^9 .

Step 2: jump by index instead. Number the living children $0 \dots \text{size}-1$ in circle order, starting from the current child. The one to remove is at index $(\text{idx} + k) \bmod \text{size}$. The mod takes care of k being huge.

Step 3: after removing index idx , where do we continue? The next child shifts down into index idx , so the new starting index is simply idx again (mod the new size, in case we removed the last one).

Step 4: the data structure question. We need a sorted collection that supports:

- “give me the element at index i ” (order statistic), and
- “delete it”,

both in $O(\log n)$. Two classic tools do this:

- a GNU **ordered_set**, which has `find_by_order(i)`, or
- a **Fenwick tree** over positions $1 \dots n$ that stores 1 for each alive child. The alive child at index i is the smallest position whose prefix sum reaches $i + 1$.

WORKED EXAMPLE

$n = 7, k = 2$

circle (by index)	$\text{idx} = (\text{idx} + k) \% \text{size}$	removed
1 2 3 4 5 6 7	$(0+2)\%7 = 2$	3
1 2 4 5 6 7	$(2+2)\%6 = 4$	6
1 2 4 5 7	$(4+2)\%5 = 1$	2
1 4 5 7	$(1+2)\%4 = 3$	7
1 4 5	$(0+2)\%3 = 2$	5
1 4	$(0+2)\%2 = 0$	1
4	$(0+2)\%1 = 0$	4

output: 3 6 2 7 5 1 4

(after a removal, idx is reduced mod the new size, e.g. $3 \% 3 = 0$)

Approach 1 • Vector with `erase`, $O(n^2)$

The same index arithmetic on a plain vector. It's correct, but each erase is $O(n)$, about $2 \cdot 10^{10}$ operations in the worst case.

```

void solve() {
    int n, k; cin >> n >> k;
    vector<int> c(n);
    iota(c.begin(), c.end(), 1);
    int idx = 0;
    while (!c.empty()) {
        idx = (idx + k) % c.size();
        cout << c[idx] << ' ';
        c.erase(c.begin() + idx);
        if (!c.empty()) idx %= c.size();
    }
    cout << '\n';
}

```

Approach 2 · Ordered set (policy-based data structure), $O(n \log n)$

```

// (1) put these lines right under #include <bits/stdc++.h> (ABOVE #define int)
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

// (2) everything below goes in place of solve()
// ordered_set: a set that also answers "give me the i-th element" in  $O(\log n)$ 
typedef tree<int, null_type, less<int>, rb_tree_tag,
            tree_order_statistics_node_update> ordered_set;

void solve() {
    int n, k; cin >> n >> k;
    ordered_set circle;
    for (int i = 1; i <= n; i++) circle.insert(i);

    int idx = 0; // index (0-based) where counting starts
    while (!circle.empty()) {
        idx = (idx + k) % circle.size(); // skip k children, wrapping around
        auto it = circle.find_by_order(idx); // the idx-th smallest = the child at idx
        cout << *it << ' ';
        circle.erase(it);
        // the child after the removed one now sits at the SAME index idx
        if (!circle.empty()) idx %= circle.size();
    }
    cout << '\n';
}

```

Approach 3 · Fenwick tree with binary descent, $O(n \log n)$

How does `kth` work? A Fenwick node `bit[pos + step]` stores the count of the block `(pos, pos + step]`. We build the answer position bit by bit, from the largest power of two down. If the whole next block still has fewer than `k` alive children, we skip it (`pos += step`) and look for the remaining `k - bit[pos]` children to its right. At the end, `pos + 1` is the position we want. That's one pass, $O(\log n)$, without an extra binary search.

```

// Fenwick tree over positions 1..n: bit stores 1 if the child is still in the circle.
// "find the idx-th alive child" = descend the Fenwick tree by powers of two.
int n, LOG;
vector<int> bit;

void add(int i, int v) { for (; i <= n; i += i & -i) bit[i] += v; }

int kth(int k) { // smallest position p with prefix_sum(p) = k (1-based k)

```

```

int pos = 0;
for (int step = LOG; step > 0; step >>= 1)
    if (pos + step <= n && bit[pos + step] < k) {
        pos += step;
        k -= bit[pos];
    }
return pos + 1;
}

void solve() {
    int k; cin >> n >> k;
    bit.assign(n + 1, 0);
    LOG = 1; while (LOG * 2 <= n) LOG *= 2;
    for (int i = 1; i <= n; i++) add(i, 1);

    int idx = 0, size = n;
    while (size > 0) {
        idx = (idx + k) % size;
        int child = kth(idx + 1);           // (idx+1)-th alive child
        cout << child << ' ';
        add(child, -1);
        size--;
        if (size) idx %= size;
    }
    cout << '\n';
}

```

KEY IDEA

“Find the k -th remaining element” = **order statistics**. Keep `ordered_set` and the Fenwick descent in your library, because they come back in many range-query problems. The Fenwick version is usually 3–5× faster.

Complexity & pitfalls

Approach	Time	Memory
Vector erase	$O(n^2)$	$O(n)$
<code>ordered_set</code>	$O(n \log n)$	$O(n)$
Fenwick descent	$O(n \log n)$	$O(n)$

WATCH OUT

- $k = 0$ means nobody is skipped, and the order is 1 2 3 ... n . The formula handles it.
- Reduce `idx + k` modulo the **current** size. Never loop k times.
- Remember the includes for `ordered_set` go **above** `#define int long long int`.

20 • Nested Ranges Check

cses.fi/problemset/task/2168 • Pattern: **sort by (left ↑, right ↓) + prefix max/min**

The task

n ranges $[x, y]$. Range A **contains** range B if $x_A \leq x_B$ and $y_B \leq y_A$. For each range, output

- line 1: 1 if it contains some other range, else 0,
- line 2: 1 if some other range contains it, else 0.

$n \leq 2 \cdot 10^5$.

Building the intuition

Step 1: two conditions at once is hard, so turn one of them into “order”. Sort ranges by left end ascending. Now, for range i , every range *before* it has a left end $\leq x_i$. So “is range i contained in some other range?” becomes: **does some earlier range reach at least as far right ($y \geq y_i$)?** That’s a running maximum.

Step 2: the tie problem. Two ranges with the **same left end**: $[2, 8]$ and $[2, 5]$. The longer one contains the shorter one, so the longer one must come **first** in our order. So we sort by **left ascending, then right descending**. Then for equal left ends, the containing range is always processed first.

Step 3: the other line, symmetric. Walk the same order **backwards**. Every range after i has left end $\geq x_i$ (and for equal lefts, a smaller right end). So “does i contain some other range?” becomes: **does some later range end at or before y_i ?** That’s a running minimum from the right.

Step 4: identical ranges. If $[3, 6]$ appears twice, each copy contains the other. A one-directional sweep only marks one of the two, so we count exact duplicates separately and set both flags for them.

WORKED EXAMPLE

Ranges (1,6) (2,4) (4,8) (3,6). Sorted: (1,6) (2,4) (3,6) (4,8)

forward (max right so far):	backward (min right so far):
(1,6): max=-inf -> inside 0	(4,8): min=+inf -> contains 0
(2,4): max=6 >= 4 -> inside 1	(3,6): min=8 > 6 -> contains 0
(3,6): max=6 >= 6 -> inside 1	(2,4): min=6 > 4 -> contains 0
(4,8): max=6 < 8 -> inside 0	(1,6): min=4 <= 6 -> contains 1
output (original order): 1 0 0 0 / 0 1 0 1	

Approach 1 • Brute force, $O(n^2)$

```
void solve() {
    int n; cin >> n;
    vector<int> x(n), y(n);
    for (int i = 0; i < n; i++) cin >> x[i] >> y[i];
    vector<int> contains(n, 0), inside(n, 0);
    for (int i = 0; i < n; i++)
```

```

    for (int j = 0; j < n; j++)
        if (i != j && x[i] <= x[j] && y[j] <= y[i]) { contains[i] = 1; inside[j] = 1; }
    print(contains); print(inside);
}

```

Approach 2 · Sort + two sweeps, $O(n \log n)$

```

void solve() {
    int n; cin >> n;
    vector<int> l(n), r(n);
    for (int i = 0; i < n; i++) cin >> l[i] >> r[i];

    // order: left end ascending; for equal left ends, the LONGER range first
    vector<int> ord(n);
    iota(ord.begin(), ord.end(), 0);
    sort(ord.begin(), ord.end(), [&](int a, int b) {
        if (l[a] != l[b]) return l[a] < l[b];
        return r[a] > r[b];
    });

    vector<int> contains(n, 0), inside(n, 0);

    // Pass 1 (left -> right): everything before me starts no later than me.
    // Someone contains me <=> one of them reaches at least as far right.
    int maxR = LLONG_MIN;
    for (int i : ord) {
        if (maxR >= r[i]) inside[i] = 1;
        maxR = max(maxR, r[i]);
    }

    // Pass 2 (right -> left): everything after me starts no earlier than me.
    // I contain someone <=> one of them ends no later than me.
    int minR = LLONG_MAX;
    for (int k = n - 1; k >= 0; k--) {
        int i = ord[k];
        if (minR <= r[i]) contains[i] = 1;
        minR = min(minR, r[i]);
    }

    // Identical ranges contain each other; the passes above see only one direction.
    map<pair<int,int>, int> cnt;
    for (int i = 0; i < n; i++) cnt[{l[i], r[i]}]++;
    for (int i = 0; i < n; i++)
        if (cnt[{l[i], r[i]}] > 1) contains[i] = inside[i] = 1;

    print(contains);
    print(inside);
}

```

Approach 3 · Reuse the counting solution, $O(n \log n)$

If we can **count** how many ranges contain each range (problem 21), then “is the count > 0 ?” answers this problem too. The code is the Fenwick solution from problem 21 with one extra step at the end. It’s heavier than needed, but it shows how the two problems relate.

```

struct Fenwick {
    // prefix sums with point updates, both  $O(\log n)$ 
    int n; vector<int> t;
    Fenwick(int n) : n(n), t(n + 1, 0) {}
    void add(int i, int v) { for (; i <= n; i += i & -i) t[i] += v; }
    int sum(int i) { int s = 0; for (; i > 0; i -= i & -i) s += t[i]; return s; }
};

void solve() {

```

```

int n; cin >> n;
vector<int> l(n), r(n);
for (int i = 0; i < n; i++) cin >> l[i] >> r[i];

// compress right ends to 1..m so they can index the Fenwick tree
vector<int> vals(r);
sort(vals.begin(), vals.end());
vals.erase(unique(vals.begin(), vals.end()), vals.end());
int m = vals.size();
vector<int> rc(n);
for (int i = 0; i < n; i++) rc[i] = lower_bound(vals.begin(), vals.end(), r[i]) - vals.begin() +
1;

vector<int> ord(n);
iota(ord.begin(), ord.end(), 0);
sort(ord.begin(), ord.end(), [&](int a, int b) {
    if (l[a] != l[b]) return l[a] < l[b];
    return r[a] > r[b];
});

vector<int> contains(n), inside(n);

// ---- inside[i]: ranges j with l_j <= l_i and r_j >= r_i. Sweep forward.
Fenwick f1(m);
for (int s = 0; s < n; ) {
    int e = s;
    while (e < n && l[ord[e]] == l[ord[s]] && r[ord[e]] == r[ord[s]]) e++;
    for (int k = s; k < e; k++) f1.add(rc[ord[k]], 1);
    for (int k = s; k < e; k++) {
        int i = ord[k];
        inside[i] = e - f1.sum(rc[i] - 1) - 1; // e ranges inserted; those with r >= r_i,
        // minus me
    }
    s = e;
}

// ---- contains[i]: ranges j with l_j >= l_i and r_j <= r_i. Sweep backward.
Fenwick f2(m);
for (int e = n; e > 0; ) {
    int s = e;
    while (s > 0 && l[ord[s - 1]] == l[ord[e - 1]] && r[ord[s - 1]] == r[ord[e - 1]]) s--;
    for (int k = s; k < e; k++) f2.add(rc[ord[k]], 1);
    for (int k = s; k < e; k++) {
        int i = ord[k];
        contains[i] = f2.sum(rc[i]) - 1; // inserted with r <= r_i, minus me
    }
    e = s;
}

// Nested Ranges Check only asks "is the count positive?"
for (auto &c : contains) c = (c > 0);
for (auto &c : inside) c = (c > 0);
print(contains);
print(inside);
}

```

Complexity & pitfalls

Approach	Time	Memory
Brute force	$O(n^2)$	$O(n)$
Sort + sweeps	$O(n \log n)$	$O(n)$
Fenwick counts	$O(n \log n)$	$O(n)$

WATCH OUT

- Sorting by (left $\uparrow$, right $\uparrow$) breaks ties the wrong way: $[2,5]$ would come before $[2,8]$, and the forward sweep would miss that $[2,8]$ contains it.
- Output in the **original input order**. Sort an index array, not the ranges themselves.
- Duplicate ranges: both copies contain each other.

21 • Nested Ranges Count

cses.fi/problemset/task/2169 · Pattern: **sort + Fenwick tree (2-D dominance counting)**

The task

Same setting as problem 20, but now print **how many** other ranges each range contains (line 1) and **how many** other ranges contain it (line 2).

- $n \leq 2 \cdot 10^5$.

Building the intuition

Step 1: start from problem 20. Sorting by (left $\uparrow$, right $\downarrow$) turned “ $x_j \leq x_i$ ” into “j comes before i”. In problem 20 we only needed to know whether *some* earlier range had $y \geq y_i$, so a running maximum was enough. Now we need **how many** earlier ranges have $y_j \geq y_i$.

Step 2: that’s a counting query on the right ends. While sweeping, keep a structure holding the right ends of all ranges processed so far, and ask: *how many of them are $\geq y_i$?* Then insert y_i . A **Fenwick tree** (binary indexed tree) indexed by the right-end value does both in $O(\log n)$, after we **compress** the right ends to $1 \dots m$, since they go up to 10^9 .

Step 3: the “contains” line is the mirror image. Sweep backwards: every range after i has $x_j \geq x_i$, and we count those with $y_j \leq y_i$, which is a plain prefix sum.

Step 4: identical ranges. Identical copies must count each other. The simplest robust method is to insert **the whole block of identical ranges first**, then query each one and subtract 1 for itself.

KEY IDEA

“Count pairs (j, i) with $x_j \leq x_i$ **and** $y_j \geq y_i$ ” is a **2-D dominance count**. The standard recipe: sort by one coordinate, sweep, and answer the other coordinate with a Fenwick tree. It’s worth memorizing.

WORKED EXAMPLE

Ranges (1,6) (2,4) (4,8) (3,6). Sorted: (1,6) (2,4) (3,6) (4,8)

forward: count earlier right ends $\geq$ mine	backward: count later right ends $\leq$ mine
(1,6): {} -> 0	(4,8): {} -> 0
(2,4): {6} -> 1	(3,6): {8} -> 0
(3,6): {6,4} -> 1 (the 6)	(2,4): {8,6} -> 0
(4,8): {6,4,6} -> 0	(1,6): {8,6,4} -> 2 (6 and 4)
output (input order): 2 0 0 0 / 0 1 0 1	

Approach 1 • Brute force, $O(n^2)$

```
void solve() {
    int n; cin >> n;
    vector<int> x(n), y(n);
    for (int i = 0; i < n; i++) cin >> x[i] >> y[i];
    vector<int> contains(n, 0), inside(n, 0);
    for (int i = 0; i < n; i++)
```

```

    for (int j = 0; j < n; j++)
        if (i != j && x[i] <= x[j] && y[j] <= y[i]) { contains[i]++; inside[j]++; }
    print(contains); print(inside);
}

```

Approach 2 · Coordinate compression + Fenwick tree, $O(n \log n)$

```

struct Fenwick {                                // prefix sums with point updates, both  $O(\log n)$ 
    int n; vector<int> t;
    Fenwick(int n) : n(n), t(n + 1, 0) {}
    void add(int i, int v) { for (; i <= n; i += i & -i) t[i] += v; }
    int sum(int i) { int s = 0; for (; i > 0; i -= i & -i) s += t[i]; return s; }
};

void solve() {
    int n; cin >> n;
    vector<int> l(n), r(n);
    for (int i = 0; i < n; i++) cin >> l[i] >> r[i];

    // compress right ends to 1..m so they can index the Fenwick tree
    vector<int> vals(r);
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end(), vals.end()), vals.end());
    int m = vals.size();
    vector<int> rc(n);
    for (int i = 0; i < n; i++) rc[i] = lower_bound(vals.begin(), vals.end(), r[i]) - vals.begin() + 1;

    vector<int> ord(n);
    iota(ord.begin(), ord.end(), 0);
    sort(ord.begin(), ord.end(), [&](int a, int b) {
        if (l[a] != l[b]) return l[a] < l[b];
        return r[a] > r[b];
    });

    vector<int> contains(n), inside(n);

    // ---- inside[i]: ranges j with  $l_j \leq l_i$  and  $r_j \geq r_i$ . Sweep forward.
    Fenwick f1(m);
    for (int s = 0; s < n; ) {
        int e = s;                                // [s, e) = block of identical ranges
        while (e < n && l[ord[e]] == l[ord[s]] && r[ord[e]] == r[ord[s]]) e++;
        for (int k = s; k < e; k++) f1.add(rc[ord[k]], 1);
        for (int k = s; k < e; k++) {
            int i = ord[k];
            inside[i] = e - f1.sum(rc[i] - 1) - 1; // e ranges inserted; those with  $r \geq r_i$ ,
            // minus me
        }
        s = e;
    }

    // ---- contains[i]: ranges j with  $l_j \geq l_i$  and  $r_j \leq r_i$ . Sweep backward.
    Fenwick f2(m);
    for (int e = n; e > 0; ) {
        int s = e;
        while (s > 0 && l[ord[s - 1]] == l[ord[e - 1]] && r[ord[s - 1]] == r[ord[e - 1]]) s--;
        for (int k = s; k < e; k++) f2.add(rc[ord[k]], 1);
        for (int k = s; k < e; k++) {
            int i = ord[k];
            contains[i] = f2.sum(rc[i]) - 1; // inserted with  $r \leq r_i$ , minus me
        }
        e = s;
    }
}

```

```

    }
    print(contains);
    print(inside);
}

```

Approach 3 · Ordered set of (right end, id), $O(n \log n)$

`ordered_set::order_of_key(v)` returns how many stored elements are $< v$, which is exactly the counting query we need. It saves you the compression step. Storing `(right, id)` pairs keeps equal right ends distinct inside the set.

```

// (1) put these lines right under #include <bits/stdc++.h> (ABOVE #define int)
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

// (2) everything below goes in place of solve()
// ordered set of (right end, id): order_of_key counts how many are strictly smaller
typedef tree<pair<int,int>, null_type, less<pair<int,int>>, rb_tree_tag,
            tree_order_statistics_node_update> ordered_set;

void solve() {
    int n; cin >> n;
    vector<int> l(n), r(n);
    for (int i = 0; i < n; i++) cin >> l[i] >> r[i];
    vector<int> ord(n);
    iota(ord.begin(), ord.end(), 0);
    sort(ord.begin(), ord.end(), [&](int a, int b) {
        if (l[a] != l[b]) return l[a] < l[b];
        return r[a] > r[b];
    });
    auto same = [&](int a, int b) { return l[a] == l[b] && r[a] == r[b]; };
    vector<int> contains(n), inside(n);

    ordered_set os; // forward sweep -> inside[]
    for (int s = 0, e; s < n; s = e) {
        for (e = s; e < n && same(ord[e], ord[s]); e++) os.insert({r[ord[e]], ord[e]});
        for (int k = s; k < e; k++) {
            int i = ord[k];
            int smaller = os.order_of_key({r[i], -1}); // right end < r_i
            inside[i] = (int)os.size() - smaller - 1;
        }
    }
    os.clear(); // backward sweep -> contains[]
    for (int e = n, s; e > 0; e = s) {
        for (s = e; s > 0 && same(ord[s - 1], ord[e - 1]); s--) os.insert({r[ord[s - 1]], ord[s - 1]});
        for (int k = s; k < e; k++) {
            int i = ord[k];
            contains[i] = os.order_of_key({r[i], LLONG_MAX}) - 1; // right end <= r_i, minus me
        }
    }
    print(contains);
    print(inside);
}

```

Complexity & pitfalls

Approach	Time	Memory
Brute force	$O(n^2)$	$O(n)$
Fenwick	$O(n \log n)$	$O(n)$
<code>ordered_set</code>	$O(n \log n)$, ~2× slower	$O(n)$

WATCH OUT

- Answers go in the **original** order, so sort indices, not the input array.
- Don't forget the tie order (right ↓) and identical ranges.

22 • Room Allocation

cses.fi/problemset/task/1164 · Pattern: **greedy by arrival + min-heap of free times**

The task

n customers with arrival day a and departure day b . Each customer needs a room for the days $a \dots b$ (inclusive). Two customers can share a room only if the first **departs strictly before** the second arrives. Find the **minimum number of rooms** and give each customer a room number.

- $n \leq 2 \cdot 10^5$.

Building the intuition

Step 1: a lower bound. If at some day d there are c customers staying, we need at least c rooms. So the answer is **at least** the maximum overlap (the sweep from problem 5).

Step 2: a greedy that meets this bound. Process customers by arrival day. When a customer arrives, if **any** room is free (its last guest left before today), reuse it; otherwise open a new room. We only open a room when every existing room is occupied on day a , so at that moment the overlap equals the number of rooms. Therefore the greedy never uses more rooms than the maximum overlap.

Step 3: how do we find a free room fast? A room is free if its last departure $< a$. It's enough to look at the room that became free **earliest**: if even that one is busy, all are. A **min-heap** of (departure day, room id) gives it in $O(\log n)$.

WORKED EXAMPLE

Customers (1,2) (2,4) (4,4)

```
day 1: customer (1,2) -> no rooms          -> open room 1   heap {(2,r1)}
day 2: customer (2,4) -> earliest free: 2 < 2? no -> open room 2 heap {(2,r1),(4,r2)}
day 4: customer (4,4) -> earliest free: 2 < 4 yes -> reuse room 1
answer: 2 rooms, assignment 1 2 1
```

Approach 1 • Brute force idea

For each customer, try every existing room and check it against all guests already in that room: $O(n^2)$. Counting the maximum overlap alone (for the number of rooms) is also $O(n^2)$ if done naively.

Approach 2 • Sort by arrival + min-heap, $O(n \log n)$

```
void solve() {
    int n; cin >> n;
    vector<array<int,3>> c(n);                // (arrival, departure, original index)
    for (int i = 0; i < n; i++) { cin >> c[i][0] >> c[i][1]; c[i][2] = i; }
    sort(c.begin(), c.end());                // handle customers in order of arrival

    // min-heap of (departure day, room) for rooms currently in use
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
    vector<int> room(n);
    int rooms = 0;
```

```

for (auto [a, b, id] : c) {
    if (!pq.empty() && pq.top().first < a) { // earliest-freed room is free before a
        int r = pq.top().second; pq.pop();
        room[id] = r;
        pq.push({b, r});
    } else { // every room is busy -> open a new one
        room[id] = ++rooms;
        pq.push({b, rooms});
    }
}
cout << rooms << '\n';
print(room);
}

```

Approach 3 · Event sweep + stack of free rooms, $O(n \log n)$

Turn each stay into two events: **arrive at a** and **free the room at $b + 1$** . Sort the events and, for equal days, process “free” before “arrive”, since a room freed by someone who left yesterday is usable today. Keep a stack of free room numbers: arriving pops one (or opens a new room), leaving pushes it back.

```

// Event sweep: a customer leaving on day b frees the room from day b+1.
// Keep a stack of free room numbers; reuse one on arrival, return it on leave.
void solve() {
    int n; cin >> n;
    vector<int> a(n), b(n);
    vector<array<int,3>> ev; // (day, type, id); type 0 = leave, 1 = arrive
    for (int i = 0; i < n; i++) {
        cin >> a[i] >> b[i];
        ev.push_back({a[i], 1, i});
        ev.push_back({b[i] + 1, 0, i}); // leave BEFORE arrivals of the same day
    }
    sort(ev.begin(), ev.end());

    vector<int> room(n), freeRooms;
    int rooms = 0;
    for (auto [day, type, id] : ev) {
        if (type == 0) freeRooms.push_back(room[id]); // room becomes free
        else if (!freeRooms.empty()) { room[id] = freeRooms.back(); freeRooms.pop_back(); }
        else room[id] = ++rooms;
    }
    cout << rooms << '\n';
    print(room);
}

```

KEY IDEA

The $b + 1$ trick converts an **inclusive** interval $[a, b]$ into a half-open one $[a, b + 1)$. With half-open intervals, “leave” and “arrive” on the same day never conflict, so sorting $(\text{day}, \text{type})$ with $\text{leave} = 0 < \text{arrive} = 1$ does the right thing automatically.

Complexity & pitfalls

Approach	Time	Memory
Brute force	$O(n^2)$	$O(n)$
Heap	$O(n \log n)$	$O(n)$
Events + stack	$O(n \log n)$	$O(n)$

WATCH OUT

- The condition is **strict**: a guest leaving on day 4 and another arriving on day 4 **cannot** share. Use $top < a$, not $\leq$.
- Print rooms in the **original** input order.

23 • Factory Machines

cses.fi/problemset/task/1620 • Pattern: **binary search on the answer**

The task

n machines work in parallel. Machine i needs k_i seconds per product. What is the **minimum time** to produce t products in total?

- $n \leq 2 \cdot 10^5$, $t \leq 10^9$, $k_i \leq 10^9$. The answer can be as large as 10^{18} .

Building the intuition

Step 1: the forward question is hard, the backward question is easy. “What is the minimum time?” is hard to compute directly. But “**can we make t products within T seconds?**” is easy: machine i makes $\lfloor T / k_i \rfloor$ products, so we add these up and compare with t .

Step 2: the answer to the easy question is monotonic. If T seconds are enough, then $T + 1$ seconds are also enough. So the answers look like

T	:	1	2	3	...	ans-1	ans	ans+1	...
enough?:		no	no	no	...	no	YES	YES	...

and we want the first YES, which binary search finds.

Step 3: bounds. $lo = 1$. For hi , the fastest machine alone makes t products in $\min(k) \cdot t$ seconds, so that is always enough. That’s at most 10^{18} , which fits in a `long long`.

Step 4: overflow. Summing $\lfloor T / k_i \rfloor$ over $2 \cdot 10^5$ machines with $T \approx 10^{18}$ can exceed 2^{63} . So we **stop summing as soon as the total reaches t** .

WORKED EXAMPLE

$k = 3\ 2\ 5$, $t = 7$

$T = 7$	:	$7/3 + 7/2 + 7/5 = 2 + 3 + 1 = 6$	< 7	not enough	
$T = 8$	:	$8/3 + 8/2 + 8/5 = 2 + 4 + 1 = 7$	≥ 7	enough	answer 8

Approach 1 • Simulation, $O(t \log n)$

Repeatedly let the machine that will finish its next product earliest make it (a min-heap of finish times). It’s correct, but $t = 10^9$ steps is too slow. It is a good checker.

```
// simulate: repeatedly let the machine that finishes its next product earliest produce it
void solve() {
    int n, t; cin >> n >> t;
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
    for (int i = 0; i < n; i++) { int k; cin >> k; pq.push({k, k}); }
    int last = 0;
    while (t-- > 0) { auto [fin, k] = pq.top(); pq.pop(); last = fin; pq.push({fin + k, k}); }
    cout << last << '\n';
}
```

Approach 2 · Binary search on the answer, $O(n \log(\min k \cdot t))$

```
void solve() {
    int n, t; cin >> n >> t;
    vector<int> k(n);
    for (auto &v : k) cin >> v;

    // how many products can all machines make together in `time` seconds? (capped at t)
    auto enough = [&](int time) {
        int made = 0;
        for (int v : k) {
            made += time / v;
            if (made >= t) return true;    // stop early -> no overflow
        }
        return false;
    };

    int lo = 1, hi = *min_element(k.begin(), k.end()) * t;    // fastest machine alone is enough
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (enough(mid)) hi = mid;        // mid works -> answer is mid or smaller
        else lo = mid + 1;                // mid fails -> answer is bigger
    }
    cout << lo << '\n';
}
```

Approach 3 · “Binary search by bits”, $O(n \cdot 60)$

An elegant way to write binary search with no off-by-one worries: build the **largest T that is not enough**, one bit at a time from high to low. We keep a bit whenever adding it still gives “not enough”. The answer is that value + 1.

```
// Same monotonic predicate, but "binary search by bits":
// build the LARGEST time that is NOT enough, one bit at a time; answer = that + 1.
void solve() {
    int n, t; cin >> n >> t;
    vector<int> k(n);
    for (auto &v : k) cin >> v;
    auto enough = [&](int time) {
        int made = 0;
        for (int v : k) { made += time / v; if (made >= t) return true; }
        return false;
    };

    int bad = 0;    // 0 seconds is never enough
    for (int bitv = 1LL << 60; bitv > 0; bitv >>= 1)
        if (!enough(bad + bitv)) bad += bitv;    // still not enough -> keep this bit
    cout << bad + 1 << '\n';
}
```

KEY IDEA

When to think “binary search on the answer”: the problem asks for a minimum or maximum, and checking a *candidate* answer is much easier than finding the answer directly. Problem 33 (Array Division) is the same pattern.

Complexity & pitfalls

Approach	Time	Memory
Simulation	$O(t \log n)$	$O(n)$
Binary search	$O(n \log 10^{18}) \approx 60n$	$O(n)$
Bit building	$O(60 n)$	$O(n)$

WATCH OUT

- Break early in the check (`made >= t`), or the sum overflows.
- `mid = lo + (hi - lo) / 2` avoids overflow of `lo + hi` near 10^{18} .

24 • Tasks and Deadlines

cses.fi/problemset/task/1630 • Pattern: **rewrite the objective, then exchange argument**

The task

n tasks, each with a duration a and a deadline d . You do them one after another in some order, starting at time 0. Finishing a task at time f earns $d - f$ (which can be negative). Maximize the total reward.

- $n \leq 2 \cdot 10^5$.

Building the intuition

Step 1: split the total reward. Total = $\sum (d_i - f_i) = \sum d_i - \sum f_i$. The first part is the same for **every** order. So the deadlines don't matter at all! We just need to **minimize the sum of finishing times**.

Step 2: minimize $\sum f$. If tasks are done in order $p_1, p_2, \dots, p_n$, then $f_{p_1} = a_{p_1}$, $f_{p_2} = a_{p_1} + a_{p_2}$, and so on. The duration of the first task is counted **n times**, the second $n - 1$ times, ..., the last once. To minimize the sum, give the largest multipliers to the **smallest durations**: sort by duration ascending. This is the *Shortest Job First* rule.

WHY IT IS CORRECT

Exchange argument. Take any order in which a longer task A comes immediately before a shorter task B. Swap them. Tasks before and after the pair finish at the same times. Inside the pair, the finishing times change from $(s + A, s + A + B)$ to $(s + B, s + A + B)$, which is smaller by $A - B > 0$. So any order that isn't sorted by duration can be improved.

WORKED EXAMPLE

Tasks (6,10) (8,15) (5,12). By duration: 5, 6, 8.

```
task (5,12): finish 5, reward 12 - 5 = 7
task (6,10): finish 11, reward 10 - 11 = -1
task (8,15): finish 19, reward 15 - 19 = -4          total 2
```

Approach 1 • Brute force, $O(n! \cdot n)$

Try every order with `next_permutation`. This only works for $n \leq 8$, but it confirms the rule.

Approach 2 • Sort by duration, $O(n \log n)$

```
void solve() {
    int n; cin >> n;
    vector<pair<int,int>> task(n);           // (duration, deadline)
    for (auto &[a, d] : task) cin >> a >> d;
    sort(task.begin(), task.end());         // shortest job first

    int time = 0, reward = 0;
    for (auto [a, d] : task) {
        time += a;                          // finishing moment of this task
    }
```

```

        reward += d - time;
    }
    cout << reward << '\n';
}

```

Approach 3 • Closed form, $O(n \log n)$

We can compute the formula from Step 2 directly: $\sum d$ minus $\sum (\text{sorted } a_i) \cdot (n - i)$.

```

// Closed form: sum(d) - sum(finish times).
// In sorted order, the i-th shortest duration is "paid" by itself and every later task.
void solve() {
    int n; cin >> n;
    vector<int> a(n);
    int sumD = 0;
    for (int i = 0; i < n; i++) { int d; cin >> a[i] >> d; sumD += d; }
    sort(a.begin(), a.end());
    int sumFinish = 0;
    for (int i = 0; i < n; i++) sumFinish += a[i] * (n - i); // counted (n - i) times
    cout << sumD - sumFinish << '\n';
}

```

Complexity & pitfalls

Approach	Time	Memory
Permutations	$O(n! \cdot n)$	$O(n)$
Sort + simulate	$O(n \log n)$	$O(n)$
Closed form	$O(n \log n)$	$O(n)$

WATCH OUT

- Sorting by deadline is the natural first guess, and it's **wrong**.
- The total can be around $-2 \cdot 10^{16}$ (large negative), so it needs 64 bits.

25 • Reading Books

cses.fi/problemset/task/1631 · Pattern: **lower bounds that are also achievable**

The task

Two people must **each** read all n books. Book i takes t_i time for either of them. They can't read the same book at the same time. What is the minimum time until both have finished everything?

- $n \leq 2 \cdot 10^5$.

Building the intuition

Step 1: two obvious lower bounds.

- Each person must read everything, so each needs at least $S = \sum t_i$ time.
- The longest book (length M) must be read twice, **one reader after the other**, which takes at least $2M$.

So the answer is at least $\max(S, 2M)$.

Step 2: the surprise is that this bound is always achievable. Two cases:

- **The longest book dominates** ($M \geq S - M$). Person A reads the long book while person B reads all the other books. B finishes those in $S - M \leq M$ time. Then they swap: B reads the long book while A reads the rest. Total: $2M$.
- **No single book dominates** ($M < S - M$). Lay the books end to end on a circular timeline of length S . A reads them in order starting at time 0. B reads the same circular sequence, but **shifted by M** . A book of length $t_i \leq M$ overlaps its own shifted copy only if the shift were less than t_i or more than $S - t_i$. Neither can happen, since $t_i \leq M$ and $M \leq S - M \leq S - t_i$. So nobody collides, and both finish at time S .

(Splitting a book into pieces isn't even necessary. The two-machine "open shop" scheduling theory shows a schedule without splitting also reaches $\max(S, 2M)$.)

WORKED EXAMPLE

Books 2 8 3: $S = 13$, $M = 8$, $2M = 16 > 13$. A reads 8 while B reads 2 + 3 (done at 5, then waits). At time 8 they swap: B reads 8 until 16, while A reads 2 and 3 (done at 13). **Answer 16.**

Approach 1 • Brute force (BFS over time steps)

For tiny inputs, simulate every second: each person picks a book with time left (or idles), and the two picks must differ. BFS finds the minimum number of seconds. This brute force knows nothing about the formula, and it agrees with it on every random test.

Approach 2 • The formula, $O(n)$

```
void solve() {
    int n; cin >> n;
    int sum = 0, mx = 0;
    for (int i = 0; i < n; i++) {
```

```

    int t; cin >> t;
    sum += t;
    mx = max(mx, t);
}
// each person needs `sum` time, and the longest book needs 2*mx (it can't be shared)
cout << max(sum, 2 * mx) << '\n';
}

```

KEY IDEA

A pattern worth recognizing: find **every reason** the answer can't be smaller (here, the total work per person and the longest single task), then show a construction that meets the largest of those bounds. Many “min-max” scheduling problems collapse to a one-liner this way.

Complexity & pitfalls

Approach	Time	Memory
BFS brute force	exponential	exponential
Formula	$O(n)$	$O(1)$

WATCH OUT

The sum can reach $2 \cdot 10^{14}$: 64-bit arithmetic is needed.

26 • Sum of Three Values

cses.fi/problemset/task/1641 · Pattern: **fix one element, reduce to Sum of Two Values**

The task

Find **three distinct positions** whose values sum to x , or print `IMPOSSIBLE`. Any valid answer is accepted.

- $n \leq 5000$, values up to 10^9 .

Building the intuition

Step 1: read the constraint. $n \leq 5000$ is a strong hint: $n^2 = 2.5 \cdot 10^7$ is fine, while $n^3 \approx 1.25 \cdot 10^{11}$ is not. So we're allowed **one loop over pairs**, but not over triples.

Step 2: reduce to a problem we already solved. Fix the first element a_i . Now we need two *other* elements summing to $x - a_i$. That's exactly **Sum of Two Values** (problem 7), which takes $O(n)$ with two pointers on a sorted array. So $O(n)$ choices $\times O(n)$ each = $O(n^2)$.

Step 3: avoid reusing positions. Sort (value, index) pairs. When i is fixed, run the two pointers only on the part **to the right of i** . Any valid triple can be written with i as its smallest-sorted element, so nothing is missed.

WORKED EXAMPLE

$x = 8$, $a = 2\ 7\ 5\ 1$, sorted $1\ 2\ 5\ 7$

```
fix 1 (need 7): l=2, r=7: 9 > 7 -> r--; l=2, r=5: 7 = 7 found -> values 1, 2, 5
positions: 4 1 3 (any order is accepted)
```

Approach 1 • Brute force, $O(n^3)$

```
void solve() {
    int n, x; cin >> n >> x;
    vector<int> a(n);
    for (auto &v : a) cin >> v;
    for (int i = 0; i < n; i++) for (int j = i + 1; j < n; j++) for (int k = j + 1; k < n; k++)
        if (a[i] + a[j] + a[k] == x) { cout << i + 1 << " " << j + 1 << " " << k + 1 << '\n';
    return; }
    cout << "IMPOSSIBLE\n";
}
```

Approach 2 • Sort + fix one + two pointers, $O(n^2)$

```
void solve() {
    int n, x; cin >> n >> x;
    vector<pair<int,int>> a(n); // (value, original position)
    for (int i = 0; i < n; i++) { cin >> a[i].first; a[i].second = i + 1; }
    sort(a.begin(), a.end());

    for (int i = 0; i < n; i++) { // fix the smallest of the three
        int need = x - a[i].first; // now it's "Sum of Two Values" on a[i+1..n-1]
        int l = i + 1, r = n - 1;
        while (l < r) {
```

```

        int s = a[l].first + a[r].first;
        if (s == need) {
            cout << a[l].second << " " << a[l].second << " " << a[r].second << '\n';
            return;
        }
        if (s < need) l++; else r--;
    }
}
cout << "IMPOSSIBLE\n";
}

```

Approach 3 · Fix the middle + growing hash map, $O(n^2)$ expected

A hash-map alternative that needs no sorting. Fix the **middle** index j and look up the first element among everything to its left: $x - a[j] - a[k]$ for each $k > j$. After finishing j , add $a[j]$ to the “left” map. The map only grows, so there’s no clearing cost.

```

// Fix the MIDDLE index j. Keep a hash map of every value to the LEFT of j,
// then for each k > j ask: "is x - a[j] - a[k] somewhere on the left?"
// The map only grows (one insert per j), so there is no clearing cost:  $O(n^2)$  lookups.
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t SEED = chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + SEED);
    }
};

void solve() {
    int n, x; cin >> n >> x;
    vector<int> a(n);
    for (auto &v : a) cin >> v;

    unordered_map<int, int, custom_hash> left; // value -> a position i < j
    left.reserve(2 * n);
    for (int j = 0; j < n; j++) {
        for (int k = j + 1; k < n; k++) {
            auto it = left.find(x - a[j] - a[k]);
            if (it != left.end()) {
                cout << it->second << " " << j + 1 << " " << k + 1 << '\n';
                return;
            }
        }
        left[a[j]] = j + 1; // a[j] can now be the first element
    }
    cout << "IMPOSSIBLE\n";
}

```

WATCH OUT

A tempting variant creates a **new** map for every i and fills it again. That’s $O(n^2 \log n)$ with a large constant (~ 2 s here), too slow. The version above builds a single map incrementally.

Complexity

Approach	Time	Memory
Brute force	$O(n^3)$	$O(n)$
Two pointers	$O(n^2)$	$O(n)$
Growing hash map	$O(n^2)$ expected	$O(n)$

27 • Sum of Four Values

cses.fi/problemset/task/1642 · Pattern: **meet in the middle on pairs**

The task

Find **four distinct positions** whose values sum to x , or print `IMPOSSIBLE`.

- $n \leq 1000$, values up to 10^9 .

Building the intuition

Step 1: the constraint again. $n = 1000$: $n^2 = 10^6$ is comfortable, and $n^3 = 10^9$ is borderline (it can pass only with a tiny constant).

Step 2: four = two + two. Split the quadruple into a **left pair** (i, j) and a **right pair** (k, l) with $i < j < k < l$. If we store all left-pair sums in a map, then for each right pair we look up $x - a_k - a_l$. That's "Sum of Two Values" where each value is itself a pair sum: *meet in the middle*.

Step 3: keep the pairs disjoint without extra checks. Loop over k as the **third** index. At that moment, the map contains only pairs (i, j) with $j < k$. So:

1. for every $l > k$, look up $x - a[k] - a[l]$ in the map,
2. then insert all pairs (i, k) with $i < k$, since k becomes usable as the second index of future left pairs.

This ordering guarantees $i < j < k < l$, so all four positions are distinct automatically.

WORKED EXAMPLE

$x = 15$, $a = 3\ 2\ 5\ 8\ 1\ 3\ 2\ 3$

$k=4$ (value 8, 1-based): pairs already in the map have $j < 4$, e.g. $(1,2)$ with sum $3+2 = 5$
 $l=7$ (value 2): need $15 - 8 - 2 = 5 \rightarrow$ found $(1,2) \rightarrow$ answer $1\ 2\ 4\ 7$ ($3+2+8+2 = 15$)

Approach 1 • Brute force, $O(n^4)$

Four nested loops, only for checking.

Approach 2 • Pair sums in a map, $O(n^2 \log n)$

```
void solve() {
    int n, x; cin >> n >> x;
    vector<int> a(n);
    for (auto &v : a) cin >> v;

    // pairSum[s] = some pair (i, j) with i < j < current "middle" index and a[i] + a[j] = s
    map<int, pair<int, int>> pairSum;
    for (int k = 0; k < n; k++) {
        // 1) use k as the THIRD index: try every fourth index l > k
        for (int l = k + 1; l < n; l++) {
            auto it = pairSum.find(x - a[k] - a[l]);
            if (it != pairSum.end()) {
                auto [i, j] = it->second;
                cout << i + 1 << " " << j + 1 << " " << k + 1 << " " << l + 1 << '\n';
            }
        }
        // Insert pairs (i, k) for i < k
        for (int i = 0; i < k; i++) {
            pairSum[a[i] + a[k]] = {i, k};
        }
    }
}
```

```

        return;
    }
}
// 2) now k becomes available as the SECOND index of future left pairs
for (int i = 0; i < k; i++) pairSum[a[i] + a[k]] = {i, k};
}
cout << "IMPOSSIBLE\n";
}

```

Approach 3 · Sort + fix two + two pointers, $O(n^3)$

Extend problem 26 by one level: fix the two smallest (i, j) and finish with two pointers. The worst case is about $n^3/6 \approx 1.7 \cdot 10^8$ very cheap steps, which runs in ~ 0.4 s here. It's simpler to write, but it depends on the constant factor.

```

// Sort, fix the two smallest, and finish with two pointers:  $O(n^3)$  but a tiny constant.
void solve() {
    int n, x; cin >> n >> x;
    vector<pair<int,int>> a(n);
    for (int i = 0; i < n; i++) { cin >> a[i].first; a[i].second = i + 1; }
    sort(a.begin(), a.end());

    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++) {
            int need = x - a[i].first - a[j].first;
            int l = j + 1, r = n - 1;
            while (l < r) {
                int s = a[l].first + a[r].first;
                if (s == need) {
                    cout << a[i].second << " " << a[j].second << " "
                        << a[l].second << " " << a[r].second << '\n';
                    return;
                }
                if (s < need) l++; else r--;
            }
        }
    cout << "IMPOSSIBLE\n";
}

```

KEY IDEA

Meet in the middle: when you need k items, split them into two halves, precompute all combinations of one half, and look up the complement for each combination of the other half. The cost goes from n^k to about $n^{k/2}$.

Complexity & pitfalls

Approach	Time	Memory
Brute force	$O(n^4)$	$O(n)$
Pair map	$O(n^2 \log n)$	$O(n^2)$
Two pointers	$O(n^3)$	$O(n)$

WATCH OUT

If you store all pairs first and look them up afterwards, the two pairs can share an index. The “insert pairs ending at k *after* querying” order makes that impossible.

28 • Nearest Smaller Values

cses.fi/problemset/task/1645 • Pattern: **monotonic stack**

The task

For each position i , find the **nearest position to the left** whose value is **strictly smaller** than a_i . Print 0 if there isn't one.

- $n \leq 2 \cdot 10^5$.

Building the intuition

Step 1: the brute force walks left until it finds something smaller. In a decreasing array, that's $O(n)$ per element, $O(n^2)$ in total.

Step 2: some elements can never be an answer again. Suppose $j < i$ and $a_j \geq a_i$. For any future position $p > i$, element i is closer to p than j is and **no larger**. So if j would work for p , i works too and is nearer. j is **useless forever**.

Step 3: keep only the useful candidates. After removing the useless ones, the remaining candidates (read left to right) have **strictly increasing** values. That's a *monotonic stack*. For a new element a_i :

1. pop every candidate with value $\geq a_i$ (they just became useless),
2. the top of the stack, if any, is the answer for i ,
3. push i .

Every index is pushed once and popped at most once, so the total work is $O(n)$.

WORKED EXAMPLE

$a = 2\ 5\ 1\ 4\ 8\ 3\ 2\ 5$ (positions 1...8)

i	a	stack before (pos:val)	pop	answer	stack after
1	2	-	-	0	1:2
2	5	1:2	-	1	1:2 2:5
3	1	1:2 2:5	2:5, 1:2	0	3:1
4	4	3:1	-	3	3:1 4:4
5	8	3:1 4:4	-	4	3:1 4:4 5:8
6	3	3:1 4:4 5:8	5:8, 4:4	3	3:1 6:3
7	2	3:1 6:3	6:3	3	3:1 7:2
8	5	3:1 7:2	-	7	3:1 7:2 8:5
output: 0 1 0 3 4 3 3 7					

Approach 1 • Brute force, $O(n^2)$

```
void solve() {
    int n; cin >> n;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];
    for (int i = 1; i <= n; i++) {
        int j = i - 1;

```

```

        while (j > 0 && a[j] >= a[i]) j--;
        cout << j << " \n"[i == n];
    }
}

```

Approach 2 · Monotonic stack, $O(n)$

```

void solve() {
    int n; cin >> n;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];

    stack<int> st; // positions; their values are strictly increasing
    for (int i = 1; i <= n; i++) {
        // anything >= a[i] can never be the answer for i or for anyone after i
        while (!st.empty() && a[st.top()] >= a[i]) st.pop();
        cout << (st.empty() ? 0 : st.top()) << " \n"[i == n];
        st.push(i);
    }
}

```

Approach 3 · Pointer jumping over previous answers, $O(n)$

Here's a stack-free version of the same idea. Start at $j = i - 1$. If $a_j \geq a_i$, then nothing between $\text{ans}[j]$ and j is smaller than a_i , so nothing there is smaller than a_i either. So we can **jump directly to $\text{ans}[j]$** . The jumps skip exactly the elements the stack would have popped, so this is also $O(n)$ amortized. A sentinel $a[0] = -\infty$ stops the jumps.

```

// "Pointer jumping": ans[i] starts at i-1 and follows the chain of answers.
// If a[j] >= a[i], nothing between ans[j] and j can help, so jump to ans[j].
void solve() {
    int n; cin >> n;
    vector<int> a(n + 1), ans(n + 1, 0);
    a[0] = LLONG_MIN; // sentinel at position 0: smaller than everything
    for (int i = 1; i <= n; i++) cin >> a[i];

    for (int i = 1; i <= n; i++) {
        int j = i - 1;
        while (a[j] >= a[i]) j = ans[j];
        ans[i] = j;
        cout << j << " \n"[i == n];
    }
}

```

KEY IDEA

The monotonic stack is the tool for “nearest greater/smaller to the left/right”, “largest rectangle in a histogram”, and “span” problems. Remember the rule: **pop what can never be the answer again**.

Complexity & pitfalls

Approach	Time	Memory
Brute force	$O(n^2)$	$O(1)$
Monotonic stack	$O(n)$	$O(n)$
Pointer jumping	$O(n)$ amortized	$O(n)$

WATCH OUT

“Strictly smaller” means we pop on $\geq$. Popping only on $>$ would leave equal values on the stack and report an equal value as “smaller”.

29 • Subarray Sums I

cses.fi/problemset/task/1660 · Pattern: **two pointers on positive numbers**

The task

Count the subarrays whose sum is exactly x . All values are **positive**.

- $n \leq 2 \cdot 10^5$, x and $a_i \leq 10^9$, $a_i \geq 1$.

Building the intuition

Step 1: positivity gives monotonicity. Adding an element to a window **increases** its sum, and removing one **decreases** it.

Step 2: sliding window. For each right end r , extend the window by a_r . If the sum is now greater than x , the window is too long, so shrink it from the left until $\text{sum} \leq x$. Because all values are positive, **at most one** left end can give sum exactly x for this r , and the shrinking stops exactly there if it exists.

Step 3: the left pointer only moves forward, so this is $O(n)$ in total.

WORKED EXAMPLE

$x = 7$, $a = 2\ 4\ 1\ 2\ 7$

```
r=0 [2]=2
r=1 [2 4]=6
r=2 [2 4 1]=7          count 1
r=3 [2 4 1 2]=9 > 7 -> drop 2 -> [4 1 2]=7      count 2
r=4 [4 1 2 7]=14 -> drop 4,1,2 -> [7]=7          count 3
```

Approach 1 • Brute force, $O(n^2)$

```
void solve() {
    int n, x; cin >> n >> x;
    vector<int> a(n);
    for (auto &v : a) cin >> v;
    int cnt = 0;
    for (int l = 0; l < n; l++) { int s = 0; for (int r = l; r < n; r++) { s += a[r]; cnt += (s == x); } }
    cout << cnt << '\n';
}
```

Approach 2 • Two pointers, $O(n)$

```
void solve() {
    int n, x; cin >> n >> x;
    vector<int> a(n);
    for (auto &v : a) cin >> v;

    int l = 0, sum = 0, cnt = 0;          // sum = a[l] + ... + a[r]
    for (int r = 0; r < n; r++) {
        sum += a[r];                     // grow the window to the right
        while (sum > x) sum -= a[l++];   // too big: shrink from the left (values are positive!)
        if (sum == x) cnt++;
    }
}
```

```

        if (sum == x) cnt++;
    }
    cout << cnt << '\n';
}

```

Approach 3 • Prefix sums + binary search, $O(n \log n)$

With positive values the prefix sums $P[0] < P[1] < \dots < P[n]$ are **strictly increasing**. A subarray $(l, r]$ has sum x exactly when $P[l] = P[r] - x$, so for each r we binary search for that value among $P[0..r-1]$. Since the P values are distinct, there's at most one match.

```

// Prefix sums are STRICTLY increasing (all a_i >= 1), so we can binary search for P[r] - x.
void solve() {
    int n, x; cin >> n >> x;
    vector<int> P(n + 1, 0);
    for (int i = 1; i <= n; i++) { int v; cin >> v; P[i] = P[i - 1] + v; }

    int cnt = 0;
    for (int r = 1; r <= n; r++)
        if (binary_search(P.begin(), P.begin() + r, P[r] - x)) cnt++; // some l < r with P[l] =
P[r] - x
    cout << cnt << '\n';
}

```

Complexity & pitfalls

Approach	Time	Memory
Brute force	$O(n^2)$	$O(n)$
Two pointers	$O(n)$	$O(n)$
Prefix + binary search	$O(n \log n)$	$O(n)$

WATCH OUT

The two-pointer method **relies on positivity**. With negative numbers (problem 30) it is simply wrong, because shrinking the window might *increase* the sum.

30 • Subarray Sums II

cses.fi/problemset/task/1661 · Pattern: **prefix sums + frequency map**

The task

Count the subarrays with sum exactly x . Values **can be negative**.

- $n \leq 2 \cdot 10^5$, $|x|, |a_i| \leq 10^9$.

Building the intuition

Step 1: why two pointers break. With negative numbers the window sum isn't monotonic, so we can't decide which pointer to move.

Step 2: rewrite the condition with prefix sums. $\text{sum}(l+1..r) = P[r] - P[l] = x \iff P[l] = P[r] - x$.

Step 3: count, don't search. For a fixed r , the number of good subarrays ending at r equals **the number of earlier prefix sums equal to $P[r] - x$** . Walk left to right and keep a frequency map of the prefix sums seen so far, including $P[0] = 0$ for the empty prefix.

WORKED EXAMPLE

$x = 7, a = 2 \ -1 \ 3 \ 5 \ -2$

```
prefix P : 0  2  1  4  9  7
r=1 P=2  need -5 : 0
r=2 P=1  need -6 : 0
r=3 P=4  need -3 : 0
r=4 P=9  need  2 : P[1]=2 seen once -> +1    (-1 3 5)
r=5 P=7  need  0 : P[0]=0 seen once -> +1    (whole array)
total 2
```

KEY IDEA

$P[r] - P[l] = x \implies \text{count earlier } P[l] = P[r] - x$. This one line solves a huge family of problems: subarrays with a given sum, a given XOR (use XOR prefixes), or a given remainder (problem 31).

Approach 1 • Brute force, $O(n^2)$

```
void solve() {
    int n, x; cin >> n >> x;
    vector<int> a(n);
    for (auto &v : a) cin >> v;
    int cnt = 0;
    for (int l = 0; l < n; l++) { int s = 0; for (int r = l; r < n; r++) { s += a[r]; cnt += (s == x); } }
    cout << cnt << '\n';
}
```

Approach 2 · Prefix sums + `map`, $O(n \log n)$

```
void solve() {
    int n, x; cin >> n >> x;
    map<int,int> freq;           // freq[s] = how many prefixes seen so far have sum s
    freq[0] = 1;                // the empty prefix P[0] = 0
    int pref = 0, cnt = 0;
    for (int i = 0; i < n; i++) {
        int v; cin >> v;
        pref += v;              // pref = P[r]
        cnt += freq[pref - x];   // every earlier P[l] = P[r] - x gives one subarray
        freq[pref]++;
    }
    cout << cnt << '\n';
}
```

Approach 3 · Sort + ranks, no hashing, $O(n \log n)$

All prefix sums are known in advance, so we can sort them once and replace each one with its **rank**. Now a plain `vector<int>` of counts works. We only need one binary search to check whether $P[r] - x$ exists among the prefix sums at all. This is deterministic, immune to anti-hash tests, and usually faster than `map`.

```
// No hashing at all: sort the prefix sums once, turn them into ranks, count with a vector.
void solve() {
    int n, x; cin >> n >> x;
    vector<int> P(n + 1, 0);
    for (int i = 1; i <= n; i++) { int v; cin >> v; P[i] = P[i - 1] + v; }

    vector<int> vals(P);
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    auto rank = [&](int v) { return lower_bound(vals.begin(), vals.end(), v) - vals.begin(); };

    vector<int> freq(vals.size(), 0);
    int cnt = 0;
    freq[rank(P[0])]++;
    for (int r = 1; r <= n; r++) {
        int want = P[r] - x;
        int k = rank(want);
        if (k < (int)vals.size() && vals[k] == want) cnt += freq[k]; // want may not exist at all
        freq[rank(P[r])]++;
    }
    cout << cnt << '\n';
}
```

Complexity & pitfalls

Approach	Time	Memory
Brute force	$O(n^2)$	$O(n)$
<code>map</code>	$O(n \log n)$	$O(n)$
Sort + ranks	$O(n \log n)$	$O(n)$

WATCH OUT

- Don't forget `freq[0] = 1` (the empty prefix). Without it, subarrays starting at index 1 are missed.
- `unordered_map` with the default hash is a known TLE on CSES for this problem. Use `map`, `rank`s, or a custom hash.

31 · Subarray Divisibility

cses.fi/problemset/task/1662 · Pattern: **prefix sums modulo n + counting equal remainders**

The task

Count the subarrays whose sum is **divisible by n** (n is also the array length).

- $n \leq 2 \cdot 10^5$, values between -10^9 and 10^9 .

Building the intuition

Step 1: same trick as problem 30. $\text{sum}(l+1..r) = P[r] - P[l]$.

Step 2: divisible means equal remainders. $P[r] - P[l] \equiv 0 \pmod n \iff P[r] \bmod n = P[l] \bmod n$. So we just count pairs of prefixes (including $P[0] = 0$) with the **same remainder**.

Step 3: remainders live in $[0, n)$, so a plain array is enough. No map is needed.

Step 4: negative numbers. In C++, $-7 \% 5$ is -2 , not 3 . Normalize with $((p \% n) + n) \% n$.

WORKED EXAMPLE

$n = 5, a = 3\ 1\ 2\ 7\ 4$

```
prefix      : 0  3  4  6 13 17
mod 5       : 0  3  4  1  3  2
remainder 3 appears twice (P[1], P[4]) -> 1 subarray: a[2..4] = 1 2 7 = 10
answer 1
```

Approach 1 · Brute force, $O(n^2)$

```
void solve() {
    int n; cin >> n;
    vector<int> a(n);
    for (auto &v : a) cin >> v;
    int cnt = 0;
    for (int l = 0; l < n; l++) { int s = 0; for (int r = l; r < n; r++) { s += a[r]; cnt += (s % n == 0); } }
    cout << cnt << '\n';
}
```

Approach 2 · Online counting, $O(n)$

```
void solve() {
    int n; cin >> n;
    vector<int> cnt(n, 0); // cnt[r] = prefixes seen so far with remainder r
    cnt[0] = 1;           // empty prefix
    int pref = 0, ans = 0;
    for (int i = 0; i < n; i++) {
        int v; cin >> v;
        pref = ((pref + v) % n + n) % n; // keep the remainder in [0, n) even for negatives
        ans += cnt[pref];                // same remainder earlier -> divisible subarray
    }
```

```

        cnt[pref]++;
    }
    cout << ans << '\n';
}

```

Approach 3 · Count first, then choose pairs, $O(n)$

If c prefixes share a remainder, every pair of them gives one subarray: $c \cdot (c - 1) / 2$ pairs. Sum that over all remainders.

```

// Count first, pair later: c prefixes with the same remainder form c*(c-1)/2 good pairs.
void solve() {
    int n; cin >> n;
    vector<int> cnt(n, 0);
    cnt[0] = 1;
    int pref = 0;
    for (int i = 0; i < n; i++) {
        int v; cin >> v;
        pref = ((pref + v) % n + n) % n;
        cnt[pref]++;
    }
    int ans = 0;
    for (int c : cnt) ans += c * (c - 1) / 2;
    cout << ans << '\n';
}

```

Complexity & pitfalls

Approach	Time	Memory
Brute force	$O(n^2)$	$O(n)$
Online	$O(n)$	$O(n)$
Pairs formula	$O(n)$	$O(n)$

WATCH OUT

- Normalize negative remainders.
- Keep `pref` reduced mod n at every step, so there's no overflow and the value can be used as an index.

32 · Distinct Values Subarrays II

cses.fi/problemset/task/2428 · Pattern: **sliding window with a distinct counter**

The task

Count the subarrays that contain **at most k distinct values**.

- $n \leq 2 \cdot 10^5, 1 \leq k \leq n$.

Building the intuition

Step 1: the property survives shrinking. If $a[l..r]$ has at most k distinct values, any sub-window has at most k too. That's the same shape as problem 16 (which was the special case "each value at most once").

Step 2: count per right end. For each r, find the smallest l such that $a[l..r]$ has $\leq k$ distinct values. Then $r - l + 1$ windows end at r.

Step 3: maintain the distinct count. Keep a frequency per value in the window. A value **enters** the distinct count when its frequency goes $0 \rightarrow 1$ and **leaves** when it goes $1 \rightarrow 0$. While the count exceeds k, move l forward.

WORKED EXAMPLE

$k = 2, a = 1\ 2\ 3\ 1\ 1$

```
r=0 [1]          distinct 1 -> +1
r=1 [1 2]        distinct 2 -> +2
r=2 [1 2 3] too many -> drop 1 -> [2 3]          -> +2
r=3 [2 3 1] too many -> drop 2 -> [3 1]          -> +2
r=4 [3 1 1]      distinct 2 -> +3
total 1+2+2+2+3 = 10
```

Approach 1 · Brute force, $O(n^2 \log n)$

```
void solve() {
    int n, k; cin >> n >> k;
    vector<int> a(n);
    for (auto &v : a) cin >> v;
    int cnt = 0;
    for (int l = 0; l < n; l++) { set<int> s; for (int r = l; r < n; r++) { s.insert(a[r]); if
    ((int)s.size() <= k) cnt++; } }
    cout << cnt << '\n';
}
```

Approach 2 · Window + `map` (map size = distinct count), $O(n \log n)$

```
void solve() {
    int n, k; cin >> n >> k;
    vector<int> a(n);
    for (auto &v : a) cin >> v;

    map<int,int> freq; // value -> count inside [l, r]
    int l = 0, total = 0;
    for (int r = 0; r < n; r++) {
```

```

    freq[a[r]]++;
    while ((int)freq.size() > k) { // too many distinct values: shrink from the left
        if (--freq[a[l]] == 0) freq.erase(a[l]);
        l++;
    }
    total += r - l + 1;           // every start in [l, r] works for this end r
}
cout << total << '\n';
}

```

Approach 3 · Compressed values + array counts, $O(n \log n)$ sort, $O(n)$ sweep

```

// Same window, but compress values first so counts live in a plain vector (faster than map)
void solve() {
    int n, k; cin >> n >> k;
    vector<int> a(n);
    for (auto &v : a) cin >> v;
    vector<int> vals(a);
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    for (auto &v : a) v = lower_bound(vals.begin(), vals.end(), v) - vals.begin();

    vector<int> freq(vals.size(), 0);
    int distinct = 0, l = 0, total = 0;
    for (int r = 0; r < n; r++) {
        if (freq[a[r]]++ == 0) distinct++;
        while (distinct > k)
            if (--freq[a[l++]] == 0) distinct--;
        total += r - l + 1;
    }
    cout << total << '\n';
}

```

NOTE

A related trick: “exactly k distinct” = $\text{atMost}(k) - \text{atMost}(k - 1)$. The same window counts both.

Complexity & pitfalls

Approach	Time	Memory
Brute force	$O(n^2 \log n)$	$O(n)$
Map window	$O(n \log n)$	$O(n)$
Compressed window	$O(n \log n)$	$O(n)$

WATCH OUT

With the map version, **erase** a key when its count drops to 0. Otherwise `freq.size()` overcounts the distinct values.

33 • Array Division

cses.fi/problemset/task/1085 • Pattern: **binary search on the answer + greedy check**

The task

Split the array into **k contiguous parts** (each non-empty) so that the **largest part sum is as small as possible**. Print that value.

- $n \leq 2 \cdot 10^5$, values up to 10^9 .

Building the intuition

Step 1: “minimize the maximum” is a signal. It’s hard to find the best split directly, but it’s easy to answer: “can every part have sum $\leq S$?”

Step 2: the greedy check. Walk left to right and keep adding elements to the current part while its sum stays $\leq S$. When the next element doesn’t fit, start a new part. This uses the **fewest possible parts** for limit S , because making a part end earlier never helps later parts. If that minimum is $\leq k$, S is feasible: extra cuts can be added anywhere, and parts only get smaller.

Step 3: monotonic. If S is feasible, $S + 1$ is too. So we binary search for the smallest feasible S .

Step 4: bounds. The answer is $\geq \max(a)$ (some part contains the largest element) and $\leq \Sigma a$ (one part containing everything).

WORKED EXAMPLE

$k = 3, a = 2\ 4\ 7\ 3\ 5$

$S = 7$: [2 4] [7] [3] ... 5 doesn't fit with 3 -> [5]	4 parts > 3	no
$S = 8$: [2 4] [7] [3 5]	3 parts	yes
answer 8		

Approach 1 • DP, $O(n^2 \cdot k)$

$dp[j][i]$ = the best maximum when splitting the first i elements into j parts. It’s correct and clear, but far too slow for the real limits. It is the checker used here.

```
// dp[j][i] = best (smallest) max-sum splitting the first i elements into j parts
void solve() {
    int n, k; cin >> n >> k;
    vector<int> a(n), P(n + 1, 0);
    for (int i = 0; i < n; i++) { cin >> a[i]; P[i + 1] = P[i] + a[i]; }
    const int INF = LLONG_MAX;
    vector<vector<int>> dp(k + 1, vector<int>(n + 1, INF));
    dp[0][0] = 0;
    for (int j = 1; j <= k; j++)
        for (int i = 1; i <= n; i++)
            for (int p = 0; p < i; p++)
                if (dp[j - 1][p] != INF) dp[j][i] = min(dp[j][i], max(dp[j - 1][p], P[i] - P[p]));
    cout << dp[k][n] << '\n';
}
```

Approach 2 · Binary search + linear greedy check, $O(n \log \Sigma a)$

```
void solve() {
    int n, k; cin >> n >> k;
    vector<int> a(n);
    for (auto &v : a) cin >> v;

    // can we cut the array into at most k pieces, each with sum <= limit?
    auto possible = [&](int limit) {
        int pieces = 1, cur = 0;
        for (int v : a) {
            if (v > limit) return false; // a single element is already too big
            if (cur + v > limit) { pieces++; cur = 0; } // close this piece, start a new one
            cur += v;
        }
        return pieces <= k; // fewer pieces can always be split more
    };

    int lo = *max_element(a.begin(), a.end()); // answer is at least the biggest element
    int hi = accumulate(a.begin(), a.end(), 0LL); // ... and at most the whole sum
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (possible(mid)) hi = mid;
        else lo = mid + 1;
    }
    cout << lo << '\n';
}
```

Approach 3 · Binary search + jumping check, $O(k \log n \cdot \log \Sigma a)$

With prefix sums, the greedy “extend the part as far as possible” becomes **one binary search**: from position p , the part ends at the last q with $P[q] \leq P[p] + S$. Each check then costs $O(k \log n)$ instead of $O(n)$, which is great when k is small.

```
// Same binary search, but the check JUMPS with prefix sums:
// from position p, the farthest end of a piece is the last q with P[q] - P[p] <= limit.
// Each check costs O(k log n) instead of O(n) -> great when k is small.
void solve() {
    int n, k; cin >> n >> k;
    vector<int> P(n + 1, 0);
    int mx = 0;
    for (int i = 1; i <= n; i++) { int v; cin >> v; P[i] = P[i - 1] + v; mx = max(mx, v); }

    auto possible = [&](int limit) {
        int p = 0;
        for (int piece = 0; piece < k && p < n; piece++)
            p = upper_bound(P.begin(), P.end(), P[p] + limit) - P.begin() - 1;
        return p == n;
    };

    int lo = mx, hi = P[n];
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (possible(mid)) hi = mid; else lo = mid + 1;
    }
    cout << lo << '\n';
}
```

KEY IDEA

Problems 23 and 33 share a skeleton: **monotone yes/no question + binary search on the number**. Other classic signals are phrases like “minimum possible maximum”, “maximum possible minimum”, and “smallest time such that...”.

Complexity & pitfalls

Approach	Time	Memory
DP	$O(n^2k)$	$O(nk)$
Binary search + linear check	$O(n \log \Sigma a)$	$O(n)$
Binary search + jumps	$O(\min(n, k \log n) \cdot \log \Sigma a)$	$O(n)$

WATCH OUT

- Start `lo` at $\max(a)$. Otherwise a single element larger than S breaks the greedy (the linear check guards against it anyway).
- Σa can reach $2 \cdot 10^{14}$, so use 64 bits.

34 • Movie Festival II

cses.fi/problemset/task/1632 • Pattern: **greedy by end time + multiset of “free at” times**

The task

Like problem 6, but now there are **k club members**. Each movie can be watched by one member, and each member watches non-overlapping movies (back-to-back is fine). Maximize the **total number of movies** watched.

- $n, k \leq 2 \cdot 10^5$.

Building the intuition

Step 1: start from problem 6. With one person, “earliest ending first” is optimal. Let’s keep processing movies **by end time**.

Step 2: for each movie, decide whether and who. A member can take the movie if they’re free by its start s (their last movie ended $\leq s$). If nobody is free, skip it: any member we could “force” onto it would have to drop a movie that ends earlier.

Step 3: if several members are free, who should take it? Give it to the member who became free **latest** but still $\leq s$. That’s the **best fit**: members who were free much earlier stay available for movies that start earlier. Assigning an early-free member to this movie would waste their earlier availability.

Step 4: data structure. A **multiset** of “free at” times, starting with k zeros. For a movie (s, e) : $it = \text{upper_bound}(s)$, then step back one. That’s the largest free time $\leq s$. Replace it with e .

WORKED EXAMPLE

$k = 2$, movies $(1,5) (8,10) (3,6) (2,5) (6,9)$. By end: $(1,5) (2,5) (3,6) (6,9) (8,10)$

```
freeAt {0,0}
(1,5) : largest <= 1 is 0 -> take {0,5} watched 1
(2,5) : largest <= 2 is 0 -> take {5,5} watched 2
(3,6) : nothing <= 3      -> skip
(6,9) : largest <= 6 is 5 -> take {5,9} watched 3
(8,10): largest <= 8 is 5 -> take {9,10} watched 4      answer 4
```

WHY IT IS CORRECT

Why best fit? Suppose members A and B are free at times $f_A < f_B \leq s$. After giving the movie to B, the free times are $\{f_A, e\}$. After giving it to A, they are $\{e, f_B\}$. Since $f_A < f_B$, our set is **pointwise at least as good** (an earlier free time is never worse). Any future plan that works in the second world works in ours.

Approach 1 • Brute force (subsets + a structural fact)

For tiny inputs, try every subset of movies. A subset can be split among k people **exactly when no moment is covered by more than k of the chosen movies**: interval graphs can be coloured with as many colours as their largest clique. It’s exponential, but it independently confirms the greedy.

Approach 2 · Multiset initialized with k free members, $O(n \log n)$

```
void solve() {
    int n, k; cin >> n >> k;
    vector<pair<int,int>> mv(n); // (end, start)
    for (auto &[e, s] : mv) cin >> s >> e;
    sort(mv.begin(), mv.end()); // earliest ending first

    multiset<int> freeAt; // when each member becomes free
    for (int i = 0; i < k; i++) freeAt.insert(0);

    int watched = 0;
    for (auto [e, s] : mv) {
        auto it = freeAt.upper_bound(s); // first member free strictly AFTER s
        if (it == freeAt.begin()) continue; // nobody is free by time s -> skip movie
        --it; // the member who became free LATEST but <= s
        freeAt.erase(it);
        freeAt.insert(e); // he watches this movie until e
        watched++;
    }
    cout << watched << '\n';
}
```

Approach 3 · Members created on demand, $O(n \log \min(n, k))$

Same greedy, but the multiset holds only members who have **already watched something**. If none of them is free and fewer than k have been used, a fresh member takes the movie. This avoids inserting k zeros (useful when k is huge compared with n).

```
// Same greedy, but members are "created" only when needed:
// the multiset holds only members who have watched something so far (size <= k).
void solve() {
    int n, k; cin >> n >> k;
    vector<pair<int,int>> mv(n);
    for (auto &[e, s] : mv) cin >> s >> e;
    sort(mv.begin(), mv.end());

    multiset<int> busyUntil;
    int watched = 0;
    for (auto [e, s] : mv) {
        auto it = busyUntil.upper_bound(s);
        if (it != busyUntil.begin()) { // someone who has watched before is free
            busyUntil.erase(prev(it));
            busyUntil.insert(e);
            watched++;
        } else if ((int)busyUntil.size() < k) { // an unused member takes it
            busyUntil.insert(e);
            watched++;
        }
    }
    cout << watched << '\n';
}
```

Complexity & pitfalls

Approach	Time	Memory
Subsets	$O(2^n \cdot n^2)$	$O(n)$
Multiset with k zeros	$O((n + k) \log k)$	$O(k)$
On-demand members	$O(n \log \min(n, k))$	$O(\min(n, k))$

WATCH OUT

- Sort by **end** time. Sorting by start time breaks the greedy.
- `lower_bound(s)` would miss a member free exactly at s . Use `upper_bound(s)` and step back.

35 • Maximum Subarray Sum II

cses.fi/problemset/task/1644 · Pattern: **prefix sums + sliding window minimum**

The task

Find the maximum sum of a subarray whose **length is between a and b** (inclusive).

- $n \leq 2 \cdot 10^5$, values between -10^9 and 10^9 .

Building the intuition

Step 1: start from problem 8 (prefix form). $\text{sum}(l+1..r) = P[r] - P[l]$. For a fixed r we want the **smallest** $P[l]$.

Step 2: the length restriction becomes a window on l . The length is $r - l$, so $a \leq r - l \leq b \iff l \in [r - b, r - a]$. As r increases by 1, this window slides right by 1: one new l enters ($r - a$) and one old l leaves ($r - b - 1$).

Step 3: so we need a sliding-window minimum. Two standard tools:

- a **multiset** of the $P[l]$ values in the window (insert, erase one, read the smallest), $O(\log n)$ each, or
- a **monotonic deque** of indices with increasing P values, $O(1)$ amortized.

Step 4: why the deque works. If $l_1 < l_2$ and $P[l_1] \geq P[l_2]$, then l_1 is never needed again. It leaves the window earlier *and* is not smaller. So we pop it from the back when l_2 arrives, which is the same “pop the useless” logic as the stack in problem 28. The front is always the minimum. When the front falls out of the window, pop it from the front.

WORKED EXAMPLE

$a = 1, b = 2, x = -1\ 3\ -2\ 5\ 3\ -5\ 2\ 2$

```
P = 0 -1 2 0 5 8 3 5 7 (P[0..8])
r=4 : l in [2,3] -> min(P[2],P[3]) = 0 -> 5 - 0 = 5
r=5 : l in [3,4] -> min(0,5) = 0 -> 8 - 0 = 8 (5 3)
... best = 8
```

Approach 1 • Brute force, $O(n^2)$

```
void solve() {
    int n, A, B; cin >> n >> A >> B;
    vector<int> x(n);
    for (auto &v : x) cin >> v;
    int best = LLONG_MIN;
    for (int l = 0; l < n; l++) { int s = 0; for (int r = l; r < n; r++) { s += x[r]; int len = r - l + 1; if (A <= len && len <= B) best = max(best, s); } }
    cout << best << '\n';
}
```

Approach 2 · Multiset window, $O(n \log n)$

```
void solve() {
    int n, A, B; cin >> n >> A >> B;
    vector<int> P(n + 1, 0);
    for (int i = 1; i <= n; i++) { int v; cin >> v; P[i] = P[i - 1] + v; }

    // sum(l+1..r) = P[r] - P[l] with A <= r - l <= B -> l in [r - B, r - A]
    multiset<int> window; // the P[l] values that are currently allowed
    int best = LLONG_MIN;
    for (int r = A; r <= n; r++) {
        window.insert(P[r - A]); // l = r - A just became allowed
        if (r - B - 1 >= 0) window.erase(window.find(P[r - B - 1])); // l = r-B-1 fell out
        best = max(best, P[r] - *window.begin()); // subtract the SMALLEST allowed prefix
    }
    cout << best << '\n';
}
```

Approach 3 · Monotonic deque, $O(n)$

```
// Sliding-window minimum with a monotonic deque:  $O(n)$  total.
void solve() {
    int n, A, B; cin >> n >> A >> B;
    vector<int> P(n + 1, 0);
    for (int i = 1; i <= n; i++) { int v; cin >> v; P[i] = P[i - 1] + v; }

    deque<int> dq; // indices l, with P[l] increasing front -> back
    int best = LLONG_MIN;
    for (int r = A; r <= n; r++) {
        int add = r - A; // new allowed l
        while (!dq.empty() && P[dq.back()] >= P[add]) dq.pop_back(); // useless from now on
        dq.push_back(add);
        if (dq.front() < r - B) dq.pop_front(); // too far left: window too long
        best = max(best, P[r] - P[dq.front()]);
    }
    cout << best << '\n';
}
```

KEY IDEA

With $b = n$ and $a = 1$, this is exactly problem 8. The sliding-window minimum (monotonic deque) is one of the most reusable ideas in the whole section. It also speeds up DP transitions of the form $dp[i] = \min$ over a window of $dp[j]$.

Complexity & pitfalls

Approach	Time	Memory
Brute force	$O(n^2)$	$O(n)$
Multiset	$O(n \log n)$	$O(n)$
Deque	$O(n)$	$O(n)$

WATCH OUT

- The answer may be negative, so start from `LLONG_MIN`, not 0.
- Erase **one** copy from the multiset: `window.erase(window.find(v))`.
- `r` starts at `a`, because shorter prefixes have no valid `l` yet.

Closing notes

NOTE

The ideas that solved all 35 problems, in one list:

1. **Sort first**: it turns “anywhere” questions into “neighbour” questions (1, 2, 3, 9, 10, 24).
2. **Greedy + exchange argument**: prove a choice is safe by swapping it into any optimal answer (2, 3, 6, 14, 24, 34).
3. **Two pointers / sliding window**: valid windows are closed under shrinking, so count $r - l + 1$ (13, 16, 29, 32).
4. **Prefix sums turn subarrays into pairs**: then count with a map or array (8, 30, 31, 35).
5. **Binary search on the answer**: when checking a candidate is easy (23, 33).
6. **Balanced BSTs (`set` / `multiset`)**: for sorted collections that keep changing (4, 14, 15, 22, 34).
7. **Monotonic stack/deque**: pop what can never be the answer again (28, 35).
8. **Order statistics**: Fenwick trees and `ordered_set` (19, 21).
9. **Change the viewpoint**: positions instead of values (11, 12), reversed time (15), product of independent choices (17).